Ruta: un lenguaje para el robot del campus

Integrantes

Juan Sebastián Arias Rueda

Sofía Bermúdez González

Sofía Rudas Sáenz

Juan José Trujillo Rendón

Profesor

John Harold Morales Vélez

Materia

Procesadores de Lenguajes

Video

https://youtu.be/OWjZQXEcsSM

Universidad Autónoma de Manizales

Manizales, Caldas

2026

Ruta: un lenguaje para el robot del campus

Introducción

En el presente trabajo se define un lenguaje destinado a un robot que transporta documentos dentro de la universidad. El propósito consiste en que las rutas del robot no deban programarse manualmente mediante un código complejo, sino que cualquier persona pueda escribirlas con facilidad. Este lenguaje recibe el nombre de Ruta y permite desplazar el robot, hacerlo girar, almacenar valores en variables, efectuar operaciones, tomar decisiones a partir de lo que detectan sus sensores y repetir instrucciones.

El documento corresponde a la Parte 1 del taller, desarrollada en papel: en ella se define Ruta mediante las herramientas formales del curso. Comprende la gramática libre de contexto, con sus no terminales, terminales, producciones y símbolo inicial, expresada en BNF y en EBNF; las derivaciones por la izquierda y por la derecha con sus respectivos árboles; el comportamiento de la gramática ante un programa mal escrito, con el punto exacto en el que lo rechaza; la corrección de las dos ambigüedades de la primera versión (expresiones y decisión anidada), con dos árboles para cada una; y el vocabulario léxico (tokens, patrones y lexemas). Las palabras clave de Ruta están en inglés, como en el boceto del enunciado. La gramática de las secciones 3 y 4 es la misma que implementa el procesador de la Parte 2, escrito en Swift (`Lexer.swift` y `Parser.swift`); por esa razón, la sección 9 describe las dos ideas tomadas de ese lenguaje que Ruta incorpora: la clasificación de los operadores según el espaciado y la precedencia declarable por el programador.

1. Diseño del lenguaje y palabras clave

Como punto de partida se toma el ejemplo del enunciado: las instrucciones se escriben una debajo de otra y Ruta no necesita un encabezado. A diferencia del boceto, cada instrucción simple termina en punto y coma (`;`): como un operador puede ser prefijo (`-x`) o postfijo (`n!`), el punto y coma deja explícito dónde termina una expresión y comienza la siguiente instrucción. Las instrucciones compuestas (`repeat`, `if`) y los bloques entre llaves no lo necesitan. La Tabla 1 reúne todas las instrucciones de Ruta.

En la columna «Forma», expr designa una expresión (un número, una variable o una operación), sentencia designa una instrucción o un bloque `{ … }`, y lo que se encuentra entre corchetes [ ] es opcional; por ejemplo, el `else` puede aparecer o no.

Tabla 1. Instrucciones de Ruta

Instrucción	Forma	Ejemplo
Avanzar	`forward expr;`	`forward 10 * speed + 5;`
Girar	`turn right;` / `turn left;`	`turn right;`
Asignación	`nombre = expr;`	`speed = 2;`
Repetir	`repeat expr times sentencia`	`repeat 4 times { turn right; }`
Decisión	`if expr then sentencia [ else sentencia ]`	`if front_sensor < 30 then stop; else forward 1;`
Bloque	`{ sentencias }`	`{ forward 1; turn left; }`
Detenerse	`stop;`	`stop;`
Volver a la base	`return base;`	`return base;`
Declarar un operador	`operator símbolo infix | prefix | postfix [ precedence n ] [ assoc left | right | none ];`	`operator ** infix precedence 80 assoc right;`
Comentarios	`// hasta el fin de la línea` `/* … */` (varias líneas)	`// head back to base`

El enunciado exige que el lenguaje contenga ciertos elementos. La Tabla 2 muestra la manera en que Ruta cumple con cada uno de ellos.

Tabla 2. Elementos exigidos por el enunciado

Elemento exigido	Forma en que se cumple
Movimientos	`forward`
Giros	`turn right`, `turn left`
Variables	`speed = 2;`
Expresiones aritméticas	`+ - * / % ( )`, menos unario `-x`, factorial `n!` y operadores declarados por el programador (por ejemplo `**`)
Comparaciones	`< <= > >= == !=`
Decisiones	`if, then, else`
Ciclos	`repeat … times`
Comentarios	`//` y `/* */`

Tabla 3. Decisiones de diseño

Decisión	Elección	Razón
Encabezado	Ninguno (sin `robot nombre`)	El programa es directamente la lista de instrucciones; el nombre del robot no interviene en el análisis (Tabla 4).
Separador de sentencias	Punto y coma `;` después de cada instrucción simple	Delimita el final de cada expresión; sin él, un operador al comienzo de la línea siguiente (por ejemplo `-x`) podría leerse como continuación de la anterior.
Bloques y decisiones	`repeat n times s`, `if c then s [else s]` y `{ … }`	El cuerpo es una sola sentencia; para varias se usa un bloque. No hay palabra de cierre: el `else` pertenece al `if` más cercano (sección 5.2).
Operadores	`+ - * / %`, `< > <= >= == !=`, prefijo `-` `+`, postfijo `!`	Las comparaciones no se encadenan: cada nivel admite una sola (`a < b < c` se rechaza).
Números	Enteros y decimales (`3.14`)	El analizador léxico reconoce `dígito+ ( . dígito+ )?`.
Comentarios	`//` y `/* */`	El analizador léxico los descarta; la gramática no los percibe.
Operadores nuevos	`operator` + fijación + precedencia	Inspirado en Swift: el programa declara símbolos y su precedencia (sección 9).
Omitido	Operadores lógicos y `/` o, retroceso	Pueden incorporarse posteriormente como extensión.

Tabla 4. Diferencias entre el boceto del enunciado y Ruta

Boceto del enunciado	Ruta	Comentario
`robot Tito`	(omitido)	Ruta no usa encabezado.
`forward 10 * speed + 5`	`forward 10 * speed + 5;`	Termina en punto y coma.
`repeat 4 { … }`	`repeat 4 times { … }`	Se agrega la palabra `times`.
`if … then … else … end`	`if … then … else …`	No hay `end`; se usan bloques `{ }` cuando se necesitan varias instrucciones.
`turn right`	`turn right;`	Igual, con punto y coma.
`stop`, `return base`	`stop;`, `return base;`	Igual, con punto y coma.
`speed`, `front_sensor`	`speed`, `front_sensor`	Identificadores (no reservados); el sensor es una variable más.
`// head back to base`	`// head back to base`	Igual; además se admite `/* */`.

2. Gramática libre de contexto G = (V, Σ, P, S)

Para definir la forma en que se escribe un programa en Ruta se emplea una gramática libre de contexto, que según lo visto en clase se formaliza como una 4-tupla G = (V, Σ, P, S) (Morales, s. f.-a; Sipser, 2013, cap. 2). Primero se presenta en notación BNF y después en EBNF, conforme lo solicita el enunciado (Morales, s. f.-b). En BNF los no terminales se escriben entre < >, el símbolo ::= se lee «se define como», | significa «o» y ε representa la cadena vacía.

Tabla 5. Elementos de la gramática

Elemento	Qué es	En Ruta
S (símbolo inicial)	El no terminal desde el cual parte todo programa	<programa>
V (no terminales)	Las categorías que se explican mediante otras reglas	Los 21 de la Tabla 6
Σ (terminales)	Lo que aparece escrito literalmente en el código (los tokens)	`forward, turn, if, {, ;,` IDENT, NUMBER, …
P (producciones)	Las reglas que establecen cómo se forma cada no terminal	Las 56 producciones de la sección 3.1

Tabla 6. Grupos de no terminales

No terminales	Qué describen
<programa>, <lista_sent>, <sent>	Estructura general: la lista de instrucciones
<sent_cerrada>, <sent_abierta>	Clasifican las sentencias según tengan o no un `if` pendiente de `else` (eliminan el «dangling else»)
<bloque>, <simple>, <giro>	Bloque entre llaves e instrucciones simples
<decl_op>, <posicion>, <opciones>, <opc_assoc>, <modo>	Declaración de operadores (sección 9)
<expr>, <igualdad>, <relacional>, <suma>, <termino>, <unario>, <postfijo>, <primario>	Expresiones por niveles de precedencia: comparaciones, aritmética, prefijos y postfijos

V (no terminales): programa, lista_sent, sent, sent_cerrada, sent_abierta, bloque, simple, giro, decl_op, posicion, opciones, opc_assoc, modo, expr, igualdad, relacional, suma, termino, unario, postfijo, primario.

Σ (terminales): ; if then else repeat times { } forward turn stop return base IDENT = right left operator OP infix prefix postfix precedence NUMBER assoc == != < > <= >= + - * / % ! ( ).

S (símbolo inicial): programa.

P: las 56 producciones de la sección 3.1 (forma compacta en 3.2). Todas tienen la forma A → α, donde A es un elemento de V y α es una cadena formada por no terminales y terminales,

por lo que se trata de una gramática libre de contexto (tipo 2).

Terminales y tokens. Cada palabra reservada y cada símbolo constituye un terminal propio: `if` y `else` son terminales distintos, de la misma manera que `+` y `*`. En cambio, IDENT y NUMBER representan una clase completa: cualquier identificador corresponde al terminal IDENT y cualquier número corresponde al terminal NUMBER. El terminal OP, que solo aparece en la declaración `operator`, representa cualquier símbolo de operador. Por esta razón, el analizador léxico entrega pares (token, lexema), y la sección 8 indica a qué terminal de la gramática corresponde cada lexema.

3. Gramática final en BNF

La notación BNF procede del informe del lenguaje ALGOL 60 (Naur, 1963) y constituye la forma estándar de describir la sintaxis de un lenguaje de programación (Louden y Lambert, 2011); además, es la empleada en las diapositivas del curso.

3.1 Versión numerada (la utilizada en las derivaciones)

P1 <programa> ::= <lista_sent>
P2 <lista_sent> ::= <sent> <lista_sent>
P3 <lista_sent> ::= ε
P4 <sent> ::= <sent_cerrada>
P5 <sent> ::= <sent_abierta>
P6 <sent_cerrada> ::= <simple> ;
P7 <sent_cerrada> ::= <bloque>
P8 <sent_cerrada> ::= if <expr> then <sent_cerrada> else <sent_cerrada>
P9 <sent_cerrada> ::= repeat <expr> times <sent_cerrada>
P10 <sent_abierta> ::= if <expr> then <sent>
P11 <sent_abierta> ::= if <expr> then <sent_cerrada> else <sent_abierta>
P12 <sent_abierta> ::= repeat <expr> times <sent_abierta>
P13 <bloque> ::= { <lista_sent> }
P14 <simple> ::= forward <expr>
P15 <simple> ::= turn <giro>
P16 <simple> ::= stop
P17 <simple> ::= return base
P18 <simple> ::= IDENT = <expr>
P19 <simple> ::= <decl_op>
P20 <giro> ::= right
P21 <giro> ::= left
P22 <decl_op> ::= operator OP <posicion> <opciones>
P23 <posicion> ::= infix
P24 <posicion> ::= prefix
P25 <posicion> ::= postfix
P26 <opciones> ::= precedence NUMBER <opc_assoc>
P27 <opciones> ::= <opc_assoc>
P28 <opc_assoc> ::= assoc <modo>
P29 <opc_assoc> ::= ε
P30 <modo> ::= left
P31 <modo> ::= right
P32 <modo> ::= IDENT
P33 <expr> ::= <igualdad>
P34 <igualdad> ::= <relacional> == <relacional>
P35 <igualdad> ::= <relacional> != <relacional>
P36 <igualdad> ::= <relacional>
P37 <relacional> ::= <suma> < <suma>
P38 <relacional> ::= <suma> > <suma>
P39 <relacional> ::= <suma> <= <suma>
P40 <relacional> ::= <suma> >= <suma>

P41 <relacional> ::= <suma>
P42 <suma> ::= <suma> + <termino>
P43 <suma> ::= <suma> - <termino>
P44 <suma> ::= <termino>
P45 <termino> ::= <termino> * <unario>
P46 <termino> ::= <termino> / <unario>
P47 <termino> ::= <termino> % <unario>
P48 <termino> ::= <unario>
P49 <unario> ::= - <unario>
P50 <unario> ::= + <unario>
P51 <unario> ::= <postfijo>
P52 <postfijo> ::= <postfijo> !
P53 <postfijo> ::= <primario>
P54 <primario> ::= NUMBER
P55 <primario> ::= IDENT
P56 <primario> ::= (<expr>)

3.2 Forma compacta (21 reglas)

Corresponde a la misma gramática, con las alternativas de cada no terminal agrupadas mediante |. Genera exactamente el mismo lenguaje.

<programa> ::= <lista_sent>
<lista_sent> ::= <sent> <lista_sent> | ε
<sent> ::= <sent_cerrada> | <sent_abierta>
<sent_cerrada> ::= <simple> ;
| <bloque>
| if <expr> then <sent_cerrada> else <sent_cerrada>
| repeat <expr> times <sent_cerrada>
<sent_abierta> ::= if <expr> then <sent>
| if <expr> then <sent_cerrada> else <sent_abierta>
| repeat <expr> times <sent_abierta>
<bloque> ::= { <lista_sent> }
<simple> ::= forward <expr>
| turn <giro>
| stop
| return base
| IDENT = <expr>
| <decl_op>
<giro> ::= right | left
<decl_op> ::= operator OP <posicion> <opciones>
<posicion> ::= infix | prefix | postfix
<opciones> ::= precedence NUMBER <opc_assoc> | <opc_assoc>
<opc_assoc> ::= assoc <modo> | ε
<modo> ::= left | right | IDENT

<expr> ::= <igualdad>
<igualdad> ::= <relacional> == <relacional>
    | <relacional> != <relacional>
    | <relacional>
<relacional> ::= <suma> < <suma>
    | <suma> > <suma>
    | <suma> <= <suma>
    | <suma> >= <suma>
    | <suma>
<suma> ::= <suma> + <termino> | <suma> - <termino> | <termino>
<termino> ::= <termino> * <unario>
    | <termino> / <unario>
    | <termino> % <unario>
    | <unario>
<unario> ::= - <unario> | + <unario> | <postfijo>
<postfijo> ::= <postfijo> ! | <primario>
<primario> ::= NUMBER | IDENT | (<expr>)

Tabla 7. Correspondencia entre la forma compacta y la versión numerada

Regla compacta	Producciones de la versión numerada (3.1)
<programa>	P1
<lista_sent>	P2–P3
<sent>	P4–P5
<sent_cerrada>	P6–P9
<sent_abierta>	P10–P12
<bloque>	P13
<simple>	P14–P19
<giro>	P20–P21
<decl_op>	P22
<posicion>	P23–P25
<opciones>	P26–P27
<opc_assoc>	P28–P29
<modo>	P30–P32
<expr>	P33
<igualdad>	P34–P36
<relacional>	P37–P41
<suma>	P42–P44

Regla compacta	Producciones de la versión numerada (3.1)
<termino>	P45–P48
<unario>	P49–P51
<postfijo>	P52–P53
<primario>	P54–P56

Las derivaciones y los árboles de la sección 6 emplean la numeración de 3.1, pues de ese modo cada paso queda asociado a su producción.

Aspectos que conviene precisar:

- <lista_sent> puede ser ε (P3), de modo que un programa o un bloque { } contenga cero o más instrucciones.
- La decisión y el ciclo se definen con dos grupos de reglas (<sent_cerrada> y <sent_abierta>) para que un `else` nunca pueda saltar por encima de un `if` sin cerrar: así se elimina el «dangling else» (sección 5.2).
- Las expresiones tienen ocho niveles (<expr>, <igualdad>, <relacional>, <suma>, <termino>, <unario>, <postfijo> y <primario>), de menor a mayor precedencia, con el fin de que cada operador se aplique en el orden correcto (sección 5.1). La suma y el producto son recursivos por la izquierda; las comparaciones no son recursivas, por lo que no se encadenan, que es también lo que exige el procesador.
- Las llaves { } y el punto y coma son terminales: en BNF no son operadores, sino los símbolos que se escriben en el programa.
- En P32 el modo de asociatividad admite un IDENT que, por restricción de contexto del procesador, debe ser `none`; esa verificación no pertenece a la gramática.

4. Gramática final en EBNF

EBNF añade a BNF operadores semejantes a los de las expresiones regulares: { } indica repetición (cero o más veces), [ ] indica opcionalidad y ( ) permite agrupar; los terminales se escriben entre comillas (Morales, s. f.-a; Wirth, 1977; ISO/IEC, 1996).

```
programa = { sent } ;
sent = simple ";" | bloque | si | repetir ;
si = "if" expr "then" sent [ "else" sent ] ;
repetir = "repeat" expr "times" sent ;
bloque = "{" { sent } "}" ;
simple = "forward" expr | "turn" ( "right" | "left" ) | "stop" | "return" "base"
      | IDENT "=" expr | decl_op ;
decl_op = "operator" OP ( "infix" | "prefix" | "postfix" )
        [ "precedence" NUMBER ] [ "assoc" ( "left" | "right" | IDENT ) ] ;
```

```
expr = relacional [ ( "==" | "!=" ) relacional ] ;
relacional = suma [ ( "<" | ">" | "<=" | ">=" ) suma ] ;
suma = termino { ( "+" | "-" ) termino } ;
termino = unario { ( "*" | "/" | "%" ) unario } ;
unario = ( "-" | "+" ) unario | postfijo ;
postfijo = primario { "!" } ;
primario = NUMBER | IDENT | "(" expr ")" ;
```

Tabla 8. Diferencias entre BNF y EBNF

En BNF	En EBNF
<lista_sent> ::= <sent> <lista_sent> | ε	{ sent }
Cinco producciones para `if` y `repeat` (P8–P12, con <sent_cerrada> y <sent_abierta>)	`si = "if" expr "then" sent [ "else" sent ]` (una sola regla)
<suma> ::= <suma> + <termino> | <suma> - <termino> | <termino>	`termino { ( "+" | "-" ) termino }`
Cuatro reglas para <relacional> con operador y una sin él	`suma [ ( "<" | … ) suma ]`
Dos reglas para <giro> (right, left)	`( "right" | "left" )`
<opciones> y <opc_assoc> con alternativa ε	`[ "precedence" NUMBER ] [ "assoc" … ]`

Diferencia entre BNF y EBNF

Ambas notaciones describen el mismo lenguaje: si un programa es válido en una, también lo es en la otra. La diferencia radica en la manera de escribirlo.

BNF solo dispone de ::=, | y la recursión; para expresar que algo es opcional o que se repite es necesario crear reglas adicionales que utilizan ε. EBNF incorpora { }, [ ] y ( ), que permiten decir lo mismo de forma más breve. Por ello, Ruta cuenta con 56 producciones en BNF (21 no terminales; 21 reglas en la forma compacta) y con 14 reglas en EBNF. Los no terminales <lista_sent>, <giro>, <posicion>, <opciones>, <opc_assoc> y <modo> desaparecen en EBNF porque quedan escritos dentro de otras reglas, y <igualdad> se fusiona con <expr>. La repetición { } se implementa directamente como un ciclo en un analizador descendente, y la asociatividad por la izquierda se conserva porque las repeticiones se aplican de izquierda a derecha, que es lo que hace `Parser.swift`.

Hay una diferencia que conviene destacar: la regla `si` de la EBNF, escrita sola, **sigue siendo ambigua** (el mismo problema del «dangling else»). La EBNF no tiene cómo expresar las categorías cerrada y abierta de la BNF; se desambigua con la regla adicional «el `else` pertenece al `if` más cercano que no tenga `else`», que es exactamente lo que hace un analizador descendente voraz y lo que formalizan <sent_cerrada> y <sent_abierta>.

Durante el desarrollo de las demás tareas del taller se advirtió que la primera versión de la gramática presentaba problemas de ambigüedad. La gramática de las secciones 3 y 4 corresponde ya a la versión corregida; en la sección 5 se expone cómo era la primera versión, qué problemas tenía y qué cambios se le realizaron.

5. Ambigüedades de la primera versión (v1) y su solución

Una gramática es ambigua si alguna cadena posee dos o más árboles de derivación distintos; para demostrarlo basta con encontrar una cadena de ese tipo (Morales, s. f.-a; Sipser, 2013, cap. 2).

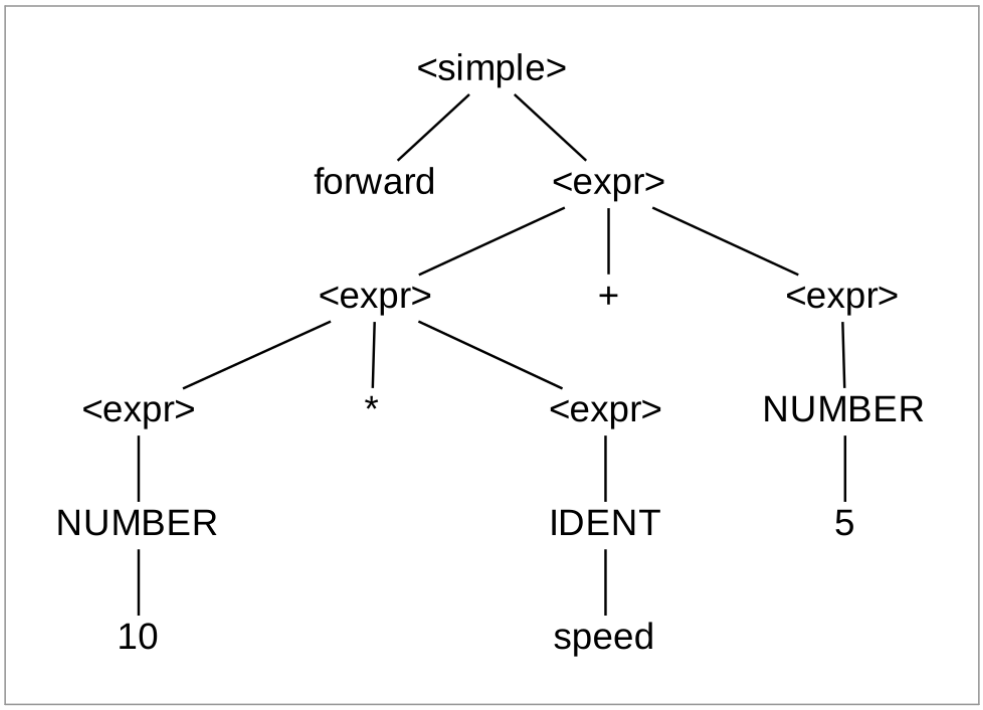
La primera versión (v1) de la gramática se redactó de la manera más directa posible y presentaba dos problemas. En las expresiones, una sola regla servía para todos los operadores (<expr> op <expr>), por lo que la gramática no indicaba cuál operación debía efectuarse primero. En las decisiones, la sentencia `if` podía llevar o no un `else` sin ninguna marca de cierre, de modo que, cuando un `if` se encuentra dentro de otro y aparece un único `else`, la gramática no determina a cuál de los dos pertenece. Solo cambian las reglas siguientes; el resto coincide con la versión final:

```
<expr> ::= <expr> + <expr> | <expr> - <expr> | <expr> * <expr> | <expr> / <expr>
   | <expr> % <expr> | <expr> < <expr> | <expr> > <expr> | <expr> <= <expr>
   | <expr> >= <expr> | <expr> == <expr> | <expr> != <expr>
   | - <expr> | + <expr> | <expr> ! | ( <expr> ) | NUMBER | IDENT
<sent> ::= if <expr> then <sent>
   | if <expr> then <sent> else <sent>
   | <simple> ; | <bloque> | repeat <expr> times <sent>
```

5.1 Ambigüedad en expresiones: forward 10 * speed + 5;

Con la v1, la misma sentencia posee dos árboles de derivación distintos (Figura 1).

Árbol A: (10 * speed) + 5



Árbol B: 10 * (speed + 5)

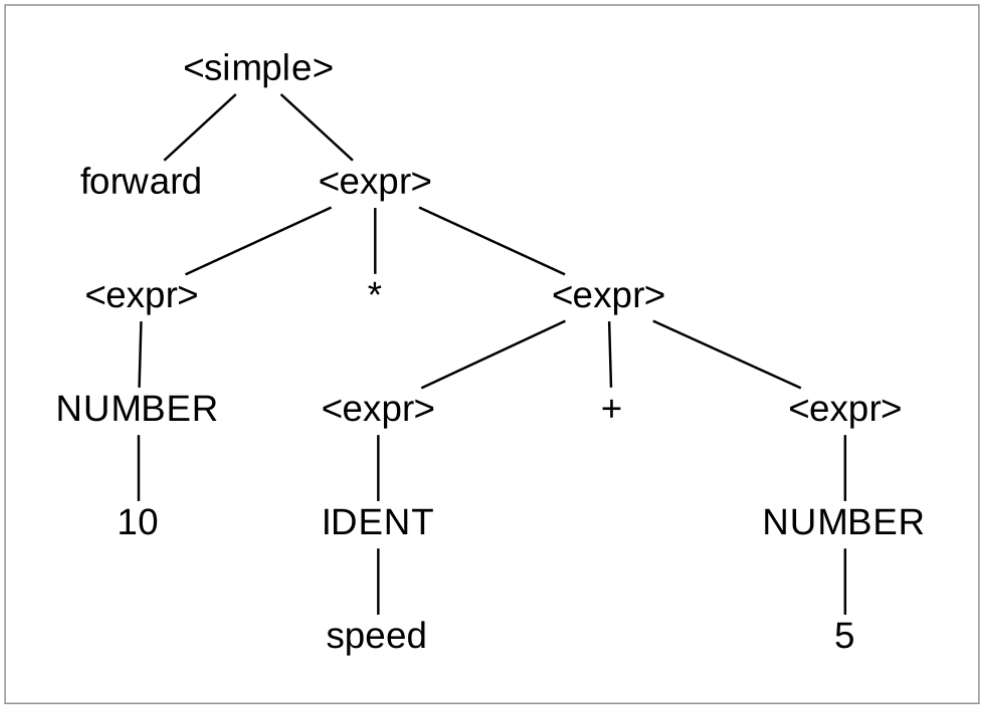


Figura 1. Dos árboles para `forward 10 * speed + 5;` con la gramática v1. En las hojas, IDENT y NUMBER llevan debajo su lexema.

Con speed = 2, el árbol A produce 25 y el árbol B produce 70. Se trata del mismo problema de J := 1 + 2 * 3 (9 frente a 7) visto en clase (Morales, s. f.-a): <expr> es recursiva por ambos

lados y nada fija la precedencia. Con la asociatividad ocurre lo mismo: 10 - 3 - 2 puede resultar en (10 − 3) − 2 = 5 o en 10 − (3 − 2) = 9; y con las comparaciones, a < b < c también admite dos agrupaciones.

En el árbol A el + queda en la parte superior, es decir, primero se multiplica. En el árbol B el * queda en la parte superior, es decir, primero se suma. Dado que una misma cadena tiene dos árboles, esta versión de la gramática es ambigua y el robot avanzaría una distancia diferente según el árbol que se elija.

Solución: un nivel por cada precedencia (<expr> → <igualdad> → <relacional> → <suma> → <termino> → <unario> → <postfijo> → <primario>), con recursión únicamente por la izquierda en la suma y el producto, que corresponde a la gramática de expresiones de las producciones P33–P56 y a la forma habitual de eliminar esta ambigüedad (Aho et al., 2007, cap. 4). La Figura 2 muestra el único árbol que esa gramática permite para la misma sentencia.

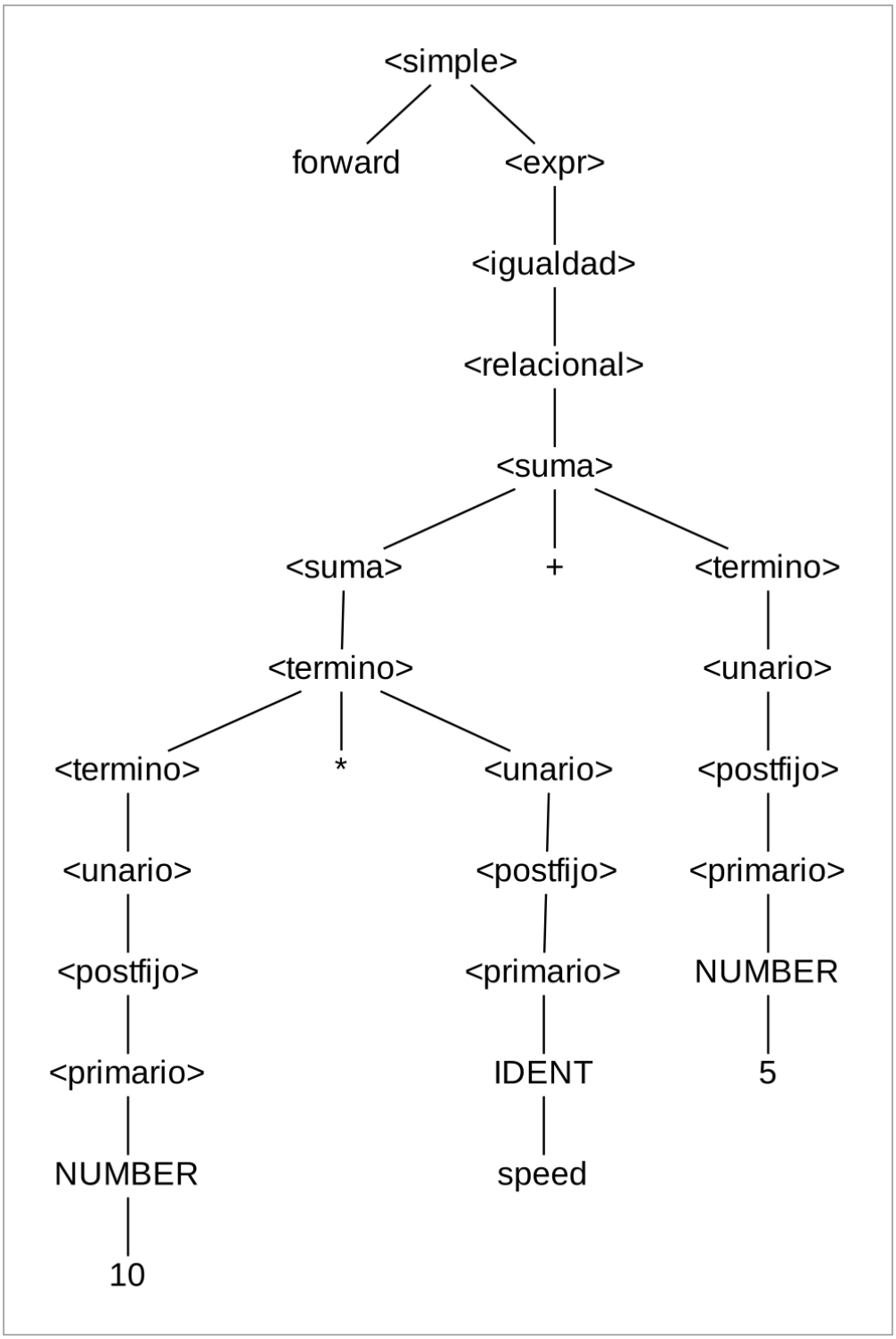


Figura 2. Árbol de `forward 10 * speed + 5;` con la gramática final: el * queda más abajo que el +.

La ambigüedad queda corregida porque el + solo aparece en <suma> y el * solo aparece en <termino>. De este modo, el * queda más abajo que el +, es decir, se efectúa primero: para calcular el valor de un nodo debe calcularse antes el valor de sus hijos. Si se desea que la

suma se realice primero, es necesario encerrarla entre paréntesis (P56); así, `10 * (speed + 5)` también tiene un único árbol (Figura 3).

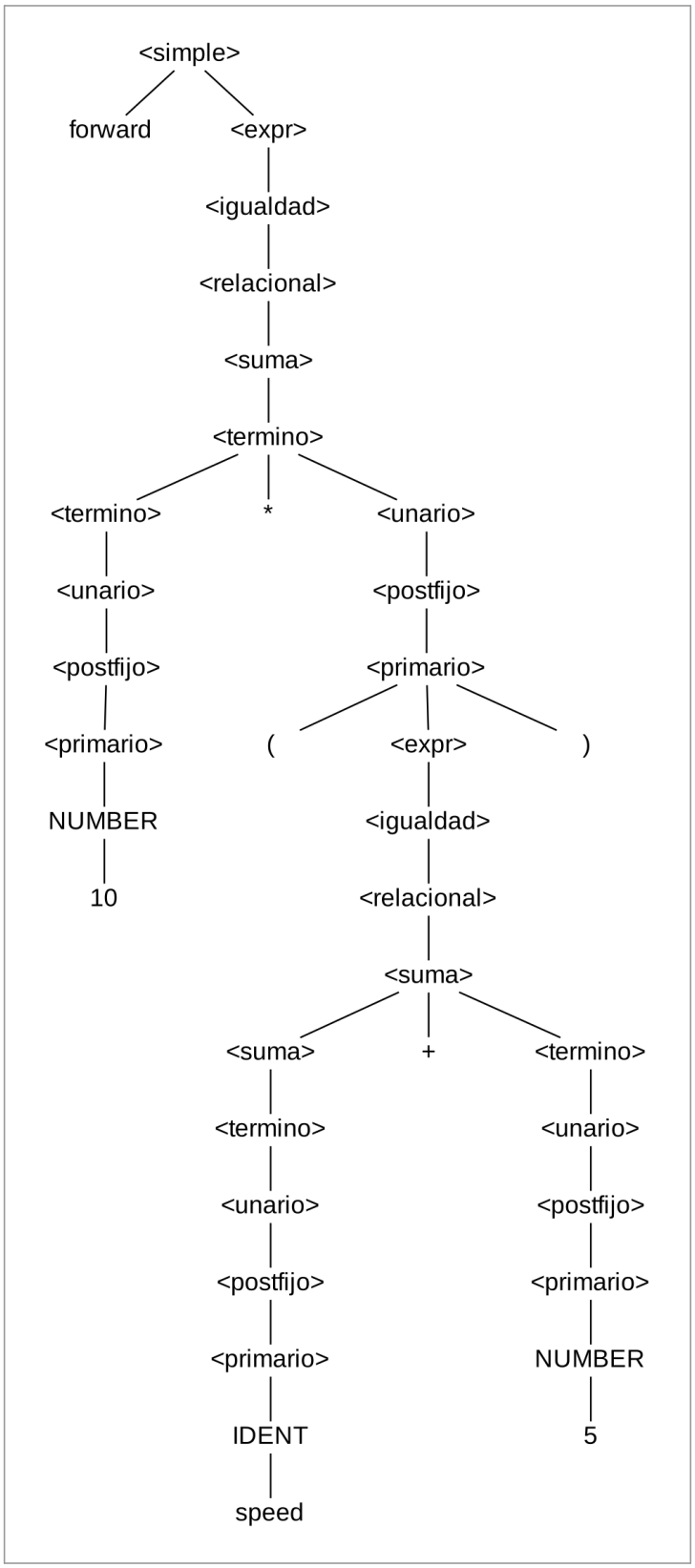
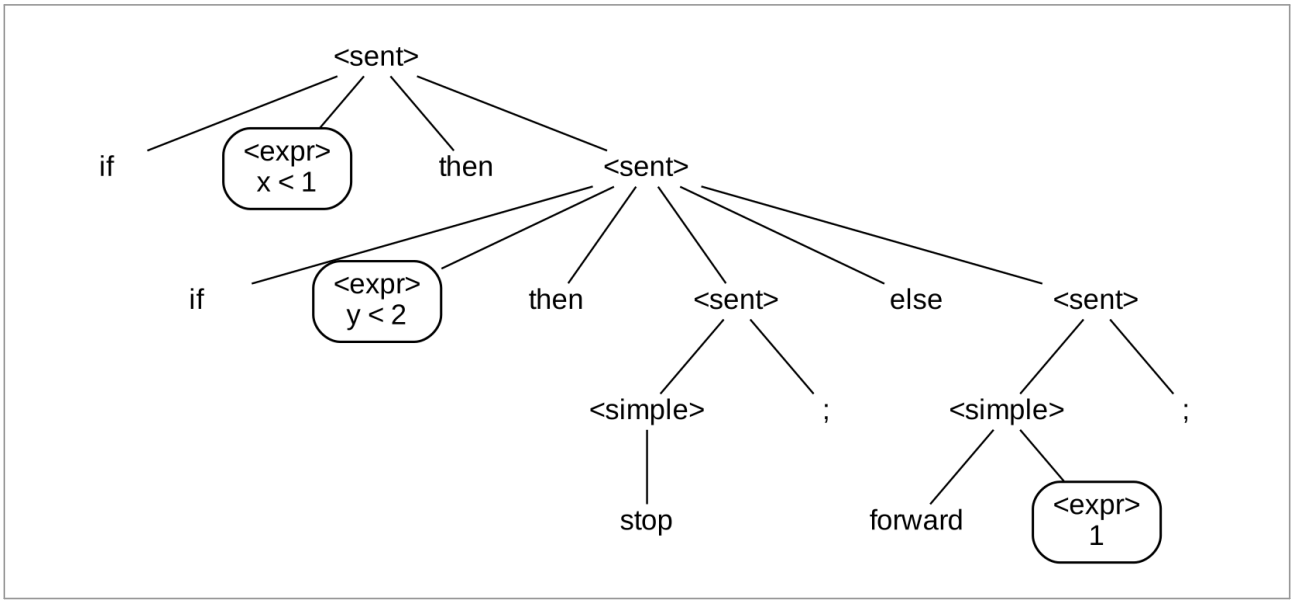

<simple>
forward
<expr>
<igualdad>
<relacional>
<suma>
<termino>
<termino>
*
<unario>
<unario>
<postfijo>
<postfijo>
<primario>
<primario>
(
<expr>
)
NUMBER
<igualdad>
10
<relacional>
<suma>
<suma>
+
<termino>
<termino>
<unario>
<unario>
<postfijo>
<postfijo>
<primario>
<primario>
NUMBER
IDENT
5
speed

Figura 3. Árbol de `forward 10 * (speed + 5);` con la gramática final: los paréntesis ubican la suma dentro de un <primario>.

5.2 Ambigüedad en decisiones anidadas (dangling else)

Cadena: `if x < 1 then if y < 2 then stop; else forward 1;`

Árbol A: el else pertenece al if interno



Árbol B: el else pertenece al if externo

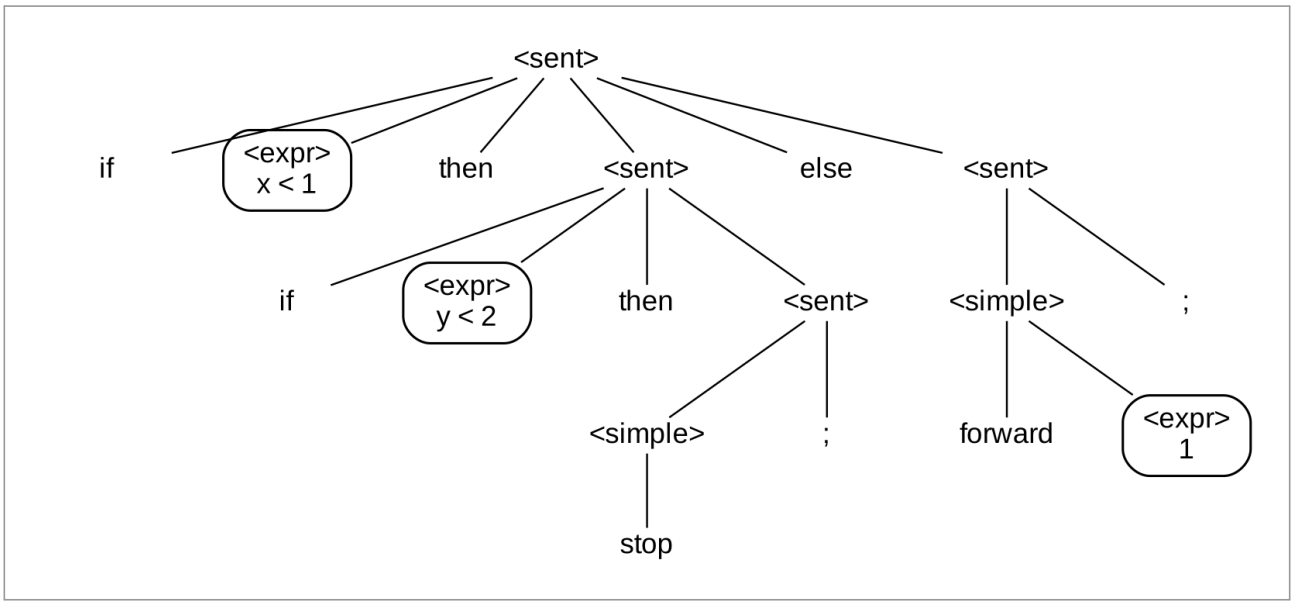


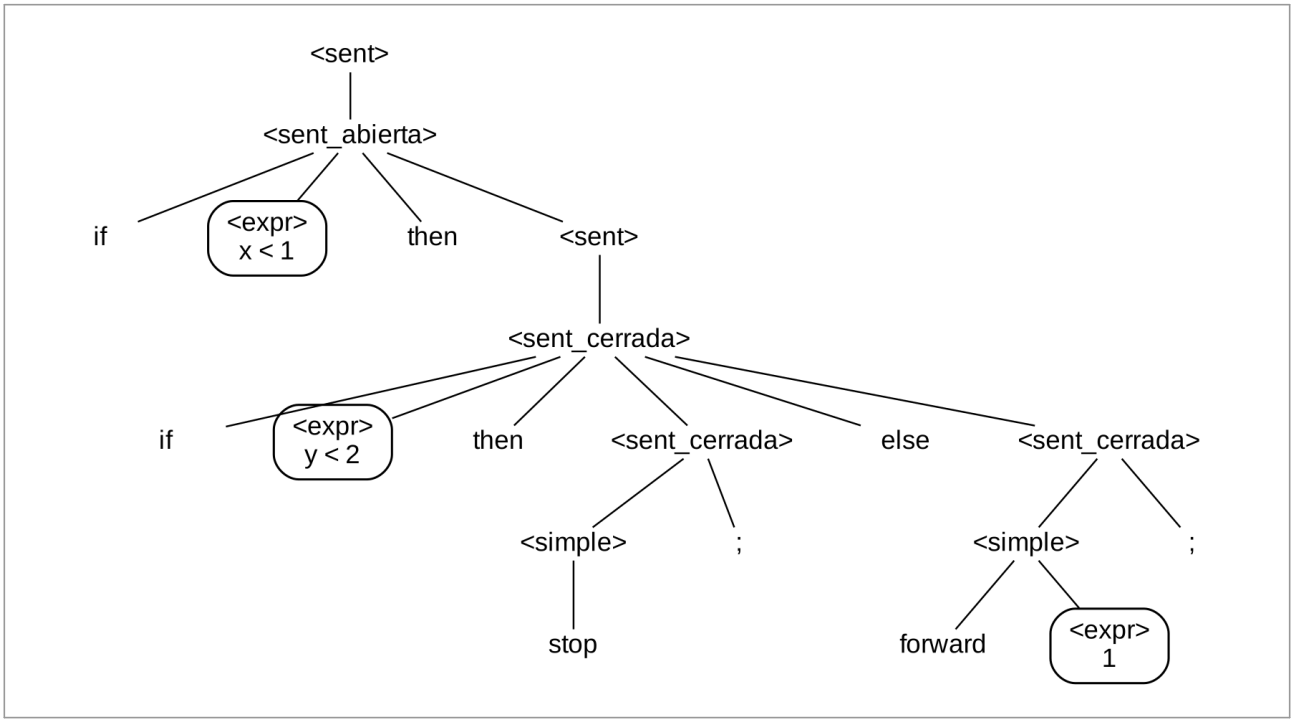
Figura 4. Dos árboles para la misma cadena con la gramática v1. Los recuadros de esquinas redondeadas son subárboles abreviados (la condición, ya descrita en el texto).

Los dos árboles tienen las mismas hojas en el mismo orden, pero el `else` depende de un `if` distinto. Para comprobarlo, se sigue hacia arriba la línea que sale del `else`: en el árbol A llega al `if` interno y en el árbol B llega al `if` externo. Con `x = 5`, el árbol A no ejecuta ninguna acción y el árbol B avanza 1: el robot se comportaría de manera distinta según cómo se lea el programa. Como la gramática permite construir la misma cadena de ambas formas, es ambigua. Se trata del mismo problema del ejercicio del if-then-else visto en clase (Morales, s. f.-a; Aho et al., 2007, cap. 4).

Solución adoptada: separar las sentencias en dos clases (Aho et al., 2007, cap. 4). Una sentencia es *cerrada* si todo `if` que contiene ya tiene su `else`, y *abierta* si contiene por lo menos un `if` sin `else`. La regla clave está en P8 y P11: entre `then` y `else` solo puede ir una sentencia cerrada, de modo que un `else` nunca puede saltar por encima de un `if` sin cerrar. Con la cadena anterior, el árbol B requeriría que `if y < 2 then stop;` fuera cerrada, pero es un `if` sin `else`, es decir, abierta. Por esta razón el árbol B ya no puede construirse y solo permanece el árbol A (Figura 5, a): el `else` se une al `if` más cercano, que es la regla utilizada por casi todos los lenguajes.

Si quien escribe el programa desea el otro significado, lo pide con un bloque: `if x < 1 then { if y < 2 then stop; } else forward 1;`. Ahora el `if` interno queda encerrado en <bloque>, que es una sentencia cerrada (P7), y el `else` pertenece al externo (Figura 5, b). Cada lectura es un programa diferente con un solo árbol.

(a) `if x < 1 then if y < 2 then stop; else forward 1;`



(b) `if x < 1 then { if y < 2 then stop; } else forward 1;`

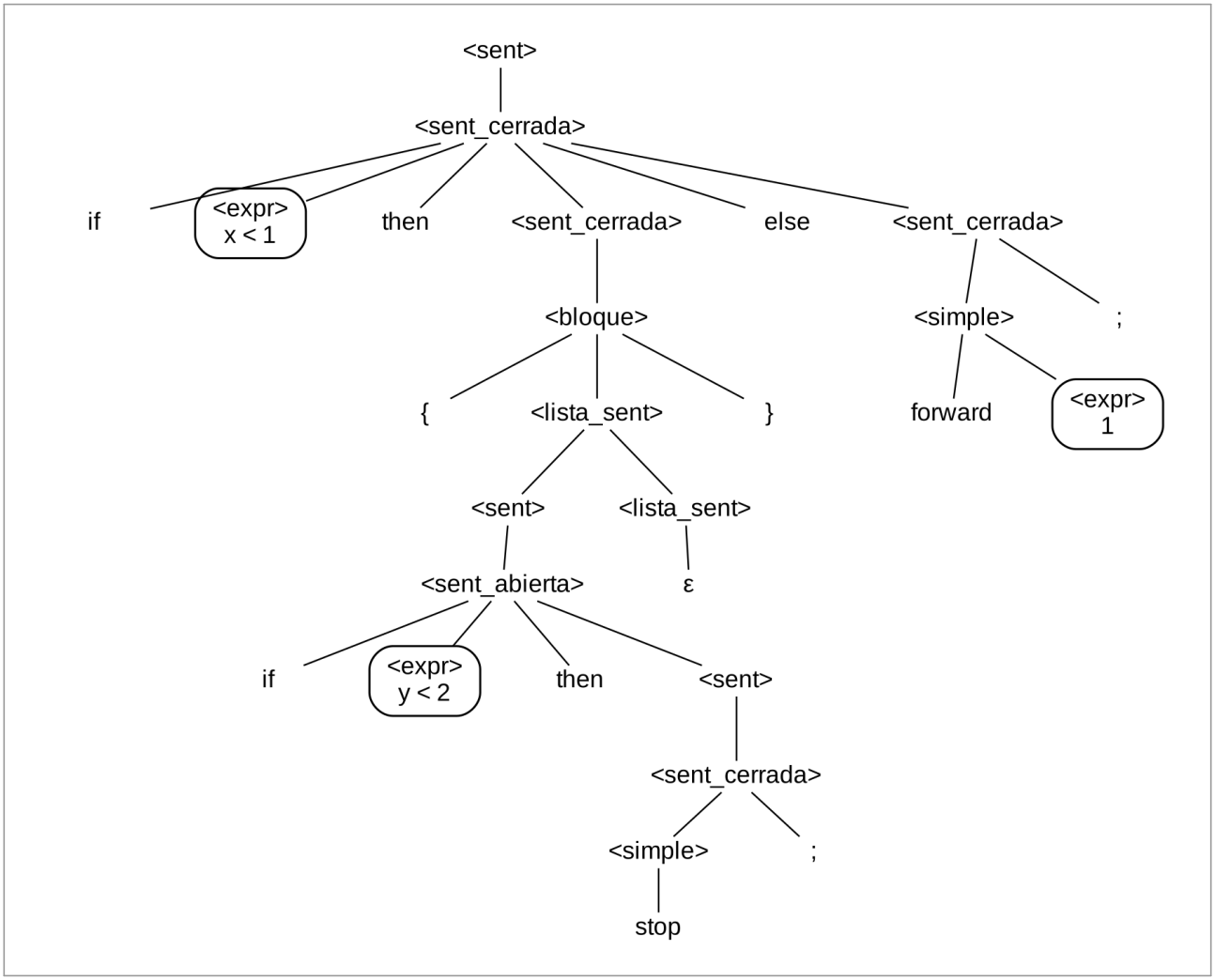


Figura 5. Con la gramática final cada lectura tiene un solo árbol: en (a) el `else` pertenece al `if` interno; en (b), al externo.

En el procesador. El analizador de la Parte 2 (`Parser.swift`, función `parseStatement`) no construye las categorías cerrada y abierta: tras analizar el `then` pregunta si sigue un `else`. Como el `if` interno se analiza primero y consume ese `else`, el externo nunca lo ve; se obtiene así el árbol de la Figura 5 (a), el mismo que fija la gramática.

5.3 Por qué la versión final no es ambigua

Fuente de ambigüedad	**Cómo se elimina**
Precedencia	+ y − solo se generan en <suma>; * / % solo en <termino>; las comparaciones, en niveles más altos. Un producto siempre queda más abajo en el árbol que una suma, salvo que existan paréntesis.
Asociatividad	<suma> y <termino> son recursivos únicamente por la izquierda, por lo que 10 - 3 - 2 se agrupa solo como (10 − 3) − 2. Las comparaciones no son recursivas: `a < b < c` no pertenece al lenguaje.
Prefijo y postfijo	<unario> y <postfijo> son niveles distintos: `-n!` solo se agrupa como −(n!).
if anidado	<sent_cerrada> y <sent_abierta>: el `else` pertenece al `if` más cercano que aún no tiene `else`; para otro significado se usa un bloque.
Límite entre sentencias	El punto y coma cierra cada sentencia simple; después de él siempre comienza otra sentencia, de modo que el límite es único.

Como comprobación adicional, se escribió un programa auxiliar que cuenta los árboles de derivación de una cadena para una gramática dada. Con la v1, `forward 10 * speed + 5;`, `x = 10 - 3 - 2;` y la cadena del `if` anidado tienen exactamente dos árboles; con la gramática final, las tres tienen uno solo, y también lo tienen las dos versiones del `if` de la Figura 5.

6. Derivaciones y árboles de derivación

Con el fin de verificar que la gramática funciona con programas reales, se deriva paso a paso un programa a partir del símbolo inicial <programa>: en cada paso se sustituye un no terminal por el lado derecho de una de sus producciones, hasta que solo permanecen terminales (Morales, s. f.-a; Aho et al., 2007, cap. 4). En la derivación por la izquierda (leftmost) siempre se sustituye el no terminal ubicado más a la izquierda; en la derivación por la derecha (rightmost), el ubicado más a la derecha. Primero se deriva un programa corto (6.1 a 6.5) y luego el programa completo del enunciado (6.6 a 6.9); en ambos casos sin omitir pasos.

6.1 Programa corto y su cadena de tokens

```
speed = 2;
forward 10 * speed + 5;
```

Tras el análisis léxico, el programa se convierte en la siguiente cadena de terminales, que es la cadena a la que debe llegar la derivación:

```
IDENT = NUMBER ; forward NUMBER * IDENT + NUMBER ;
```

Notación. En las derivaciones, el lexema de cada IDENT y NUMBER se escribe entre corchetes, por ejemplo IDENT[speed] y NUMBER[2]. Los no terminales se distinguen por los signos < > y, junto a cada paso, se indica la producción aplicada (numeración de la sección 3.1).

6.2 Derivación por la izquierda (leftmost)

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent_cerrada> <lista_sent>	P4
4	=> <simple> ; <lista_sent>	P6
5	=> IDENT[speed] = <expr> ; <lista_sent>	P18
6	=> IDENT[speed] = <igualdad> ; <lista_sent>	P33
7	=> IDENT[speed] = <relacional> ; <lista_sent>	P36
8	=> IDENT[speed] = <suma> ; <lista_sent>	P41
9	=> IDENT[speed] = <termino> ; <lista_sent>	P44
10	=> IDENT[speed] = <unario> ; <lista_sent>	P48
11	=> IDENT[speed] = <postfijo> ; <lista_sent>	P51
12	=> IDENT[speed] = <primario> ; <lista_sent>	P53
13	=> IDENT[speed] = NUMBER[2] ; <lista_sent>	P54
14	=> IDENT[speed] = NUMBER[2] ; <sent> <lista_sent>	P2
15	=> IDENT[speed] = NUMBER[2] ; <sent_cerrada> <lista_sent>	P4
16	=> IDENT[speed] = NUMBER[2] ; <simple> ; <lista_sent>	P6
17	=> IDENT[speed] = NUMBER[2] ; forward <expr> ; <lista_sent>	P14
18	=> IDENT[speed] = NUMBER[2] ; forward <igualdad> ; <lista_sent>	P33
19	=> IDENT[speed] = NUMBER[2] ; forward <relacional> ; <lista_sent>	P36
20	=> IDENT[speed] = NUMBER[2] ; forward <suma> ; <lista_sent>	P41
21	=> IDENT[speed] = NUMBER[2] ; forward <suma> + <termino> ; <lista_sent>	P42
22	=> IDENT[speed] = NUMBER[2] ; forward <termino> + <termino> ; <lista_sent>	P44
23	=> IDENT[speed] = NUMBER[2] ; forward <termino> * <unario> + <termino> ; <lista_sent>	P45
24	=> IDENT[speed] = NUMBER[2] ; forward <unario> * <unario> + <termino> ; <lista_sent>	P48
25	=> IDENT[speed] = NUMBER[2] ; forward <postfijo> * <unario> + <termino> ; <lista_sent>	P51
26	=> IDENT[speed] = NUMBER[2] ; forward <primario> * <unario> + <termino> ; <lista_sent>	P53
27	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * <unario> + <termino> ; <lista_sent>	P54
28	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * <postfijo> + <termino> ; <lista_sent>	P51
29	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * <primario> + <termino> ; <lista_sent>	P53
30	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + <termino> ; <lista_sent>	P55
31	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + <unario> ; <lista_sent>	P48
32	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + <postfijo> ; <lista_sent>	P51
33	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + <primario> ; <lista_sent>	P53
34	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; <lista_sent>	P54
35	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P3

6.3 Derivación por la derecha (rightmost)

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent> <sent> <lista_sent>	P2
4	=> <sent> <sent>	P3
5	=> <sent> <sent_cerrada>	P4
6	=> <sent> <simple> ;	P6
7	=> <sent> forward <expr> ;	P14
8	=> <sent> forward <igualdad> ;	P33
9	=> <sent> forward <relacional> ;	P36
10	=> <sent> forward <suma> ;	P41
11	=> <sent> forward <suma> + <termino> ;	P42
12	=> <sent> forward <suma> + <unario> ;	P48
13	=> <sent> forward <suma> + <postfijo> ;	P51
14	=> <sent> forward <suma> + <primario> ;	P53
15	=> <sent> forward <suma> + NUMBER[5] ;	P54
16	=> <sent> forward <termino> + NUMBER[5] ;	P44
17	=> <sent> forward <termino> * <unario> + NUMBER[5] ;	P45
18	=> <sent> forward <termino> * <postfijo> + NUMBER[5] ;	P51
19	=> <sent> forward <termino> * <primario> + NUMBER[5] ;	P53
20	=> <sent> forward <termino> * IDENT[speed] + NUMBER[5] ;	P55
21	=> <sent> forward <unario> * IDENT[speed] + NUMBER[5] ;	P48
22	=> <sent> forward <postfijo> * IDENT[speed] + NUMBER[5] ;	P51
23	=> <sent> forward <primario> * IDENT[speed] + NUMBER[5] ;	P53
24	=> <sent> forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P54
25	=> <sent_cerrada> forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P4
26	=> <simple> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P6
27	=> IDENT[speed] = <expr> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P18
28	=> IDENT[speed] = <igualdad> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P33
29	=> IDENT[speed] = <relacional> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P36
30	=> IDENT[speed] = <suma> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P41
31	=> IDENT[speed] = <termino> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P44
32	=> IDENT[speed] = <unario> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P48
33	=> IDENT[speed] = <postfijo> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P51
34	=> IDENT[speed] = <primario> ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P53
35	=> IDENT[speed] = NUMBER[2] ; forward NUMBER[10] * IDENT[speed] + NUMBER[5] ;	P54

6.4 Qué cambia entre las dos derivaciones y qué se mantiene

Las dos derivaciones constan de 35 pasos, aplican las mismas producciones el mismo número de veces y llegan al mismo programa; lo único que varía es el orden. En la derivación por la izquierda, la asignación `speed = 2;` queda completa antes de comenzar con `forward`. En la derivación por la derecha, primero se completa `forward 10 * speed + 5;` y la asignación queda para el final.

Ambas conducen al mismo árbol (sección 6.5). Esto es lo esperado: cada paso agrega al árbol el nodo que se sustituye junto con sus hijos, y el orden de los pasos no modifica la producción empleada en cada nodo. Que la derivación por la izquierda y la derivación por la derecha sean diferentes es normal y no guarda relación con la ambigüedad.

Cabe aclarar que el hecho de que estas dos derivaciones lleguen al mismo árbol no demuestra que la gramática no sea ambigua; solo muestra un árbol posible para esta cadena. Una gramática es ambigua cuando una misma cadena tiene dos árboles distintos o, lo que es equivalente, dos derivaciones por la izquierda distintas (Sipser, 2013, cap. 2). Que la versión final no lo sea se justifica en la sección 5.3; lo observado aquí es consistente con ello.

6.5 Árboles de derivación del programa corto

El árbol puede construirse por etapas siguiendo cualquiera de las dos derivaciones. Los no terminales que aún no han sido sustituidos aparecen con borde punteado.

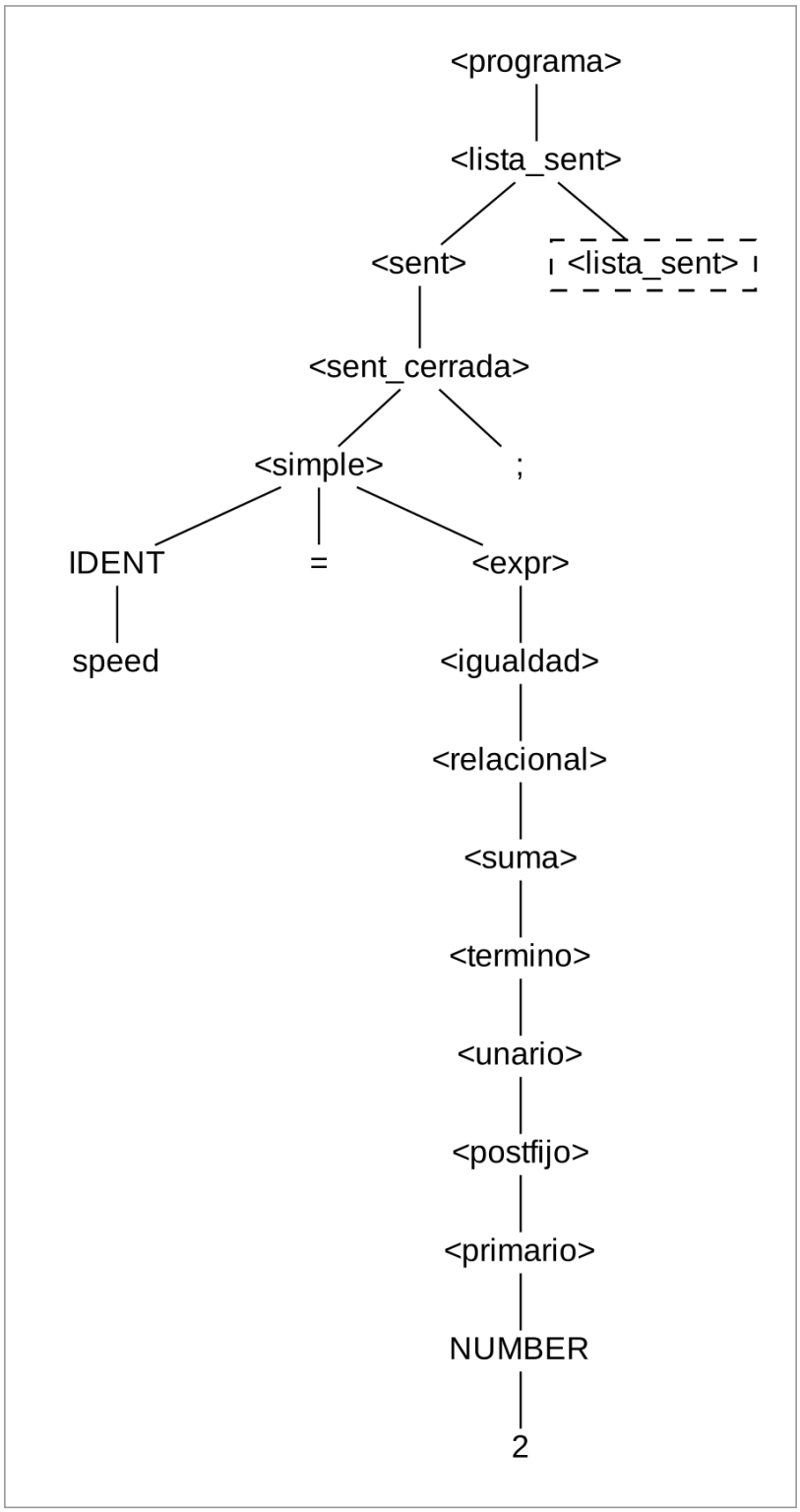


Figura 6. Por la izquierda, después del paso 13: la asignación `speed = 2;` ya está derivada.

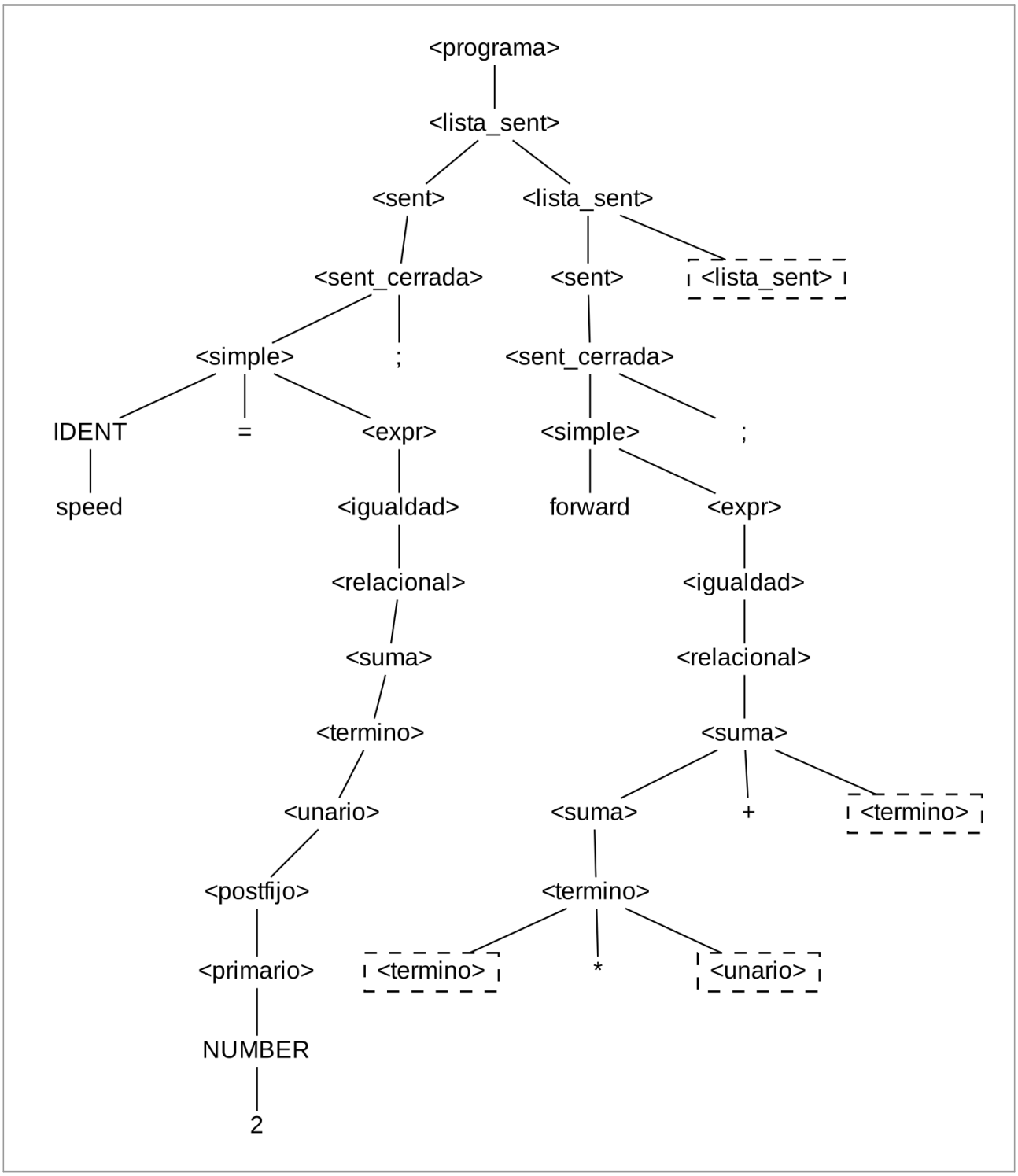


Figura 7. Por la izquierda, después del paso 23: `forward` con la multiplicación y la suma en proceso.

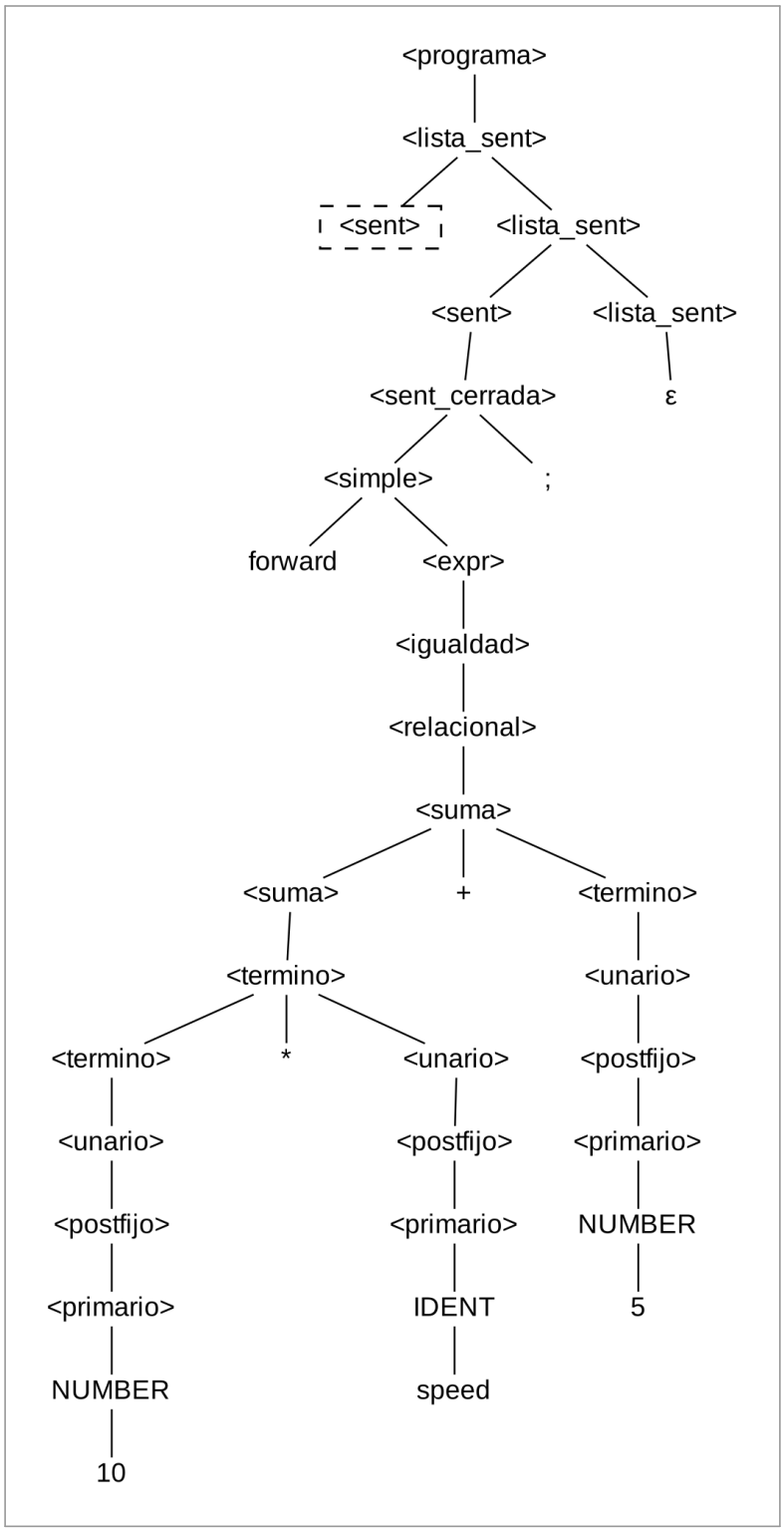


Figura 8. Por la derecha, después del paso 24: `forward 10 * speed + 5;` ya está derivado y la asignación permanece pendiente.

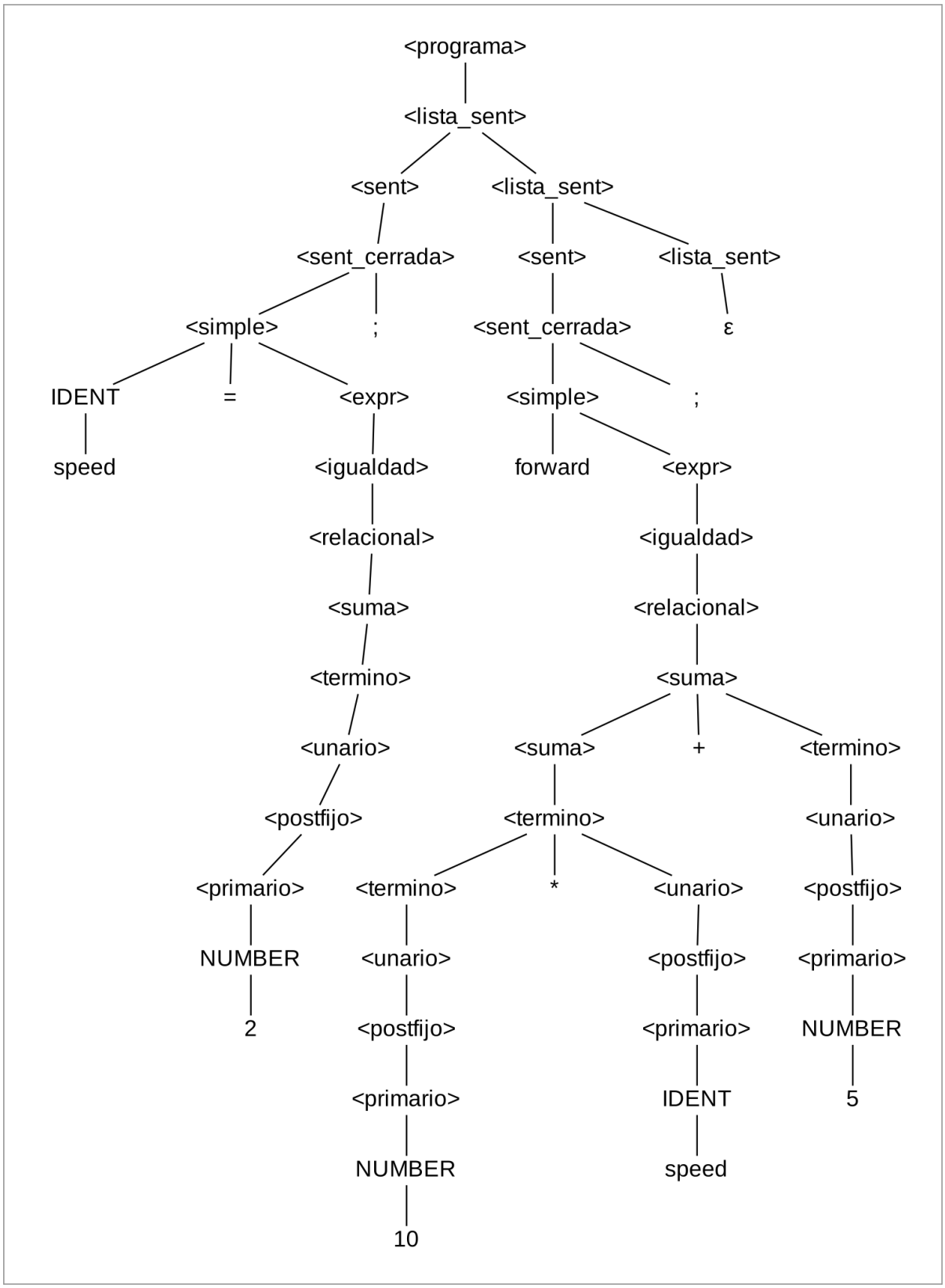

<programa>
<lista_sent>
<sent>
<lista_sent>
<sent_cerrada>
<sent>
<lista_sent>
<simple>
;
<sent_cerrada>
ε
IDENT
=
<expr>
<simple>
;
speed
<igualdad>
forward
<expr>
<relacional>
<igualdad>
<suma>
<relacional>
<termino>
<suma>
<unario>
<suma>
+
<termino>
<postfijo>
<termino>
<unario>
<primario>
<termino>
*
<unario>
<postfijo>
NUMBER
<unario>
<postfijo>
<primario>
2
<postfijo>
<primario>
NUMBER
<primario>
IDENT
5
NUMBER
speed
10

Figura 9. Árbol completo del programa corto. Es el mismo al terminar la derivación por la izquierda (paso 35) y la derivación por la derecha (paso 35).

6.6 Programa completo del enunciado

Corresponde al programa del boceto del enunciado, escrito en Ruta:

```
speed = 2;
repeat 4 times {
    forward 10 * speed + 5;
    turn right;
}
if front_sensor < 30 then
    stop;
else
    forward 1;
// head back to base
return base;
```

El comentario es eliminado por el analizador léxico antes del análisis sintáctico (sección 8). La cadena de terminales que debe derivarse es la siguiente:

```
IDENT = NUMBER ; repeat NUMBER times { forward NUMBER * IDENT + NUMBER ; turn right ; } if IDENT < NUMBER then stop ; else forward NUMBER ; return base ;
```

Abreviaturas. Para que cada línea quepa en la página, la parte de la forma sentencial que ya contiene solo terminales se reemplaza por una abreviatura: α1, α2, … al inicio en la derivación por la izquierda y ω1, ω2, … al final en la derivación por la derecha. Las abreviaturas únicamente ocultan terminales que ya fueron derivados: no se omite ningún paso ni ningún no terminal, y cada abreviatura comienza a usarse en la primera línea en que la forma sentencial deja de caber en la página.

Tabla 9. Abreviaturas utilizadas en las derivaciones del programa completo

Izquierda	Significa	Derecha	Significa
α1	IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times {	ω1	if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;
α2	α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; }	ω2	repeat NUMBER[4] times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1

6.7 Programa completo: derivación por la izquierda (leftmost)

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent_cerrada> <lista_sent>	P4
4	=> <simple> ; <lista_sent>	P6
5	=> IDENT[speed] = <expr> ; <lista_sent>	P18
6	=> IDENT[speed] = <igualdad> ; <lista_sent>	P33
7	=> IDENT[speed] = <relacional> ; <lista_sent>	P36
8	=> IDENT[speed] = <suma> ; <lista_sent>	P41
9	=> IDENT[speed] = <termino> ; <lista_sent>	P44
10	=> IDENT[speed] = <unario> ; <lista_sent>	P48
11	=> IDENT[speed] = <postfijo> ; <lista_sent>	P51
12	=> IDENT[speed] = <primario> ; <lista_sent>	P53
13	=> IDENT[speed] = NUMBER[2] ; <lista_sent>	P54
14	=> IDENT[speed] = NUMBER[2] ; <sent> <lista_sent>	P2
15	=> IDENT[speed] = NUMBER[2] ; <sent_cerrada> <lista_sent>	P4
16	=> IDENT[speed] = NUMBER[2] ; repeat <expr> times <sent_cerrada> <lista_sent>	P9
17	=> IDENT[speed] = NUMBER[2] ; repeat <igualdad> times <sent_cerrada> <lista_sent>	P33
18	=> IDENT[speed] = NUMBER[2] ; repeat <relacional> times <sent_cerrada> <lista_sent>	P36
19	=> IDENT[speed] = NUMBER[2] ; repeat <suma> times <sent_cerrada> <lista_sent>	P41
20	=> IDENT[speed] = NUMBER[2] ; repeat <termino> times <sent_cerrada> <lista_sent>	P44
21	=> IDENT[speed] = NUMBER[2] ; repeat <unario> times <sent_cerrada> <lista_sent>	P48
22	=> IDENT[speed] = NUMBER[2] ; repeat <postfijo> times <sent_cerrada> <lista_sent>	P51
23	=> IDENT[speed] = NUMBER[2] ; repeat <primario> times <sent_cerrada> <lista_sent>	P53
24	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times <sent_cerrada> <lista_sent>	P54
25	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times <bloque> <lista_sent>	P7
26	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { <lista_sent> } <lista_sent>	P13
27	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { <sent> <lista_sent> } <lista_sent>	P2
28	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { <sent_cerrada> <lista_sent> } <lista_sent>	P4
29	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { <simple> ; <lista_sent> } <lista_sent>	P6
30	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { forward <expr> ; <lista_sent> } <lista_sent>	P14
31	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { forward <igualdad> ; <lista_sent> } <lista_sent>	P33
32	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { forward <relacional> ; <lista_sent> } <lista_sent>	P36
33	=> IDENT[speed] = NUMBER[2] ; repeat NUMBER[4] times { forward <suma> ; <lista_sent> } <lista_sent>	P41
34	=> α1 forward <suma> + <termino> ; <lista_sent> } <lista_sent>	P42
35	=> α1 forward <termino> + <termino> ; <lista_sent> } <lista_sent>	P44
36	=> α1 forward <termino> * <unario> + <termino> ; <lista_sent> } <lista_sent>	P45
37	=> α1 forward <unario> * <unario> + <termino> ; <lista_sent> } <lista_sent>	P48
38	=> α1 forward <postfijo> * <unario> + <termino> ; <lista_sent> } <lista_sent>	P51
39	=> α1 forward <primario> * <unario> + <termino> ; <lista_sent> } <lista_sent>	P53
40	=> α1 forward NUMBER[10] * <unario> + <termino> ; <lista_sent> } <lista_sent>	P54
41	=> α1 forward NUMBER[10] * <postfijo> + <termino> ; <lista_sent> } <lista_sent>	P51
42	=> α1 forward NUMBER[10] * <primario> + <termino> ; <lista_sent> } <lista_sent>	P53
43	=> α1 forward NUMBER[10] * IDENT[speed] + <termino> ; <lista_sent> } <lista_sent>	P55
44	=> α1 forward NUMBER[10] * IDENT[speed] + <unario> ; <lista_sent> } <lista_sent>	P48
45	=> α1 forward NUMBER[10] * IDENT[speed] + <postfijo> ; <lista_sent> } <lista_sent>	P51
46	=> α1 forward NUMBER[10] * IDENT[speed] + <primario> ; <lista_sent> } <lista_sent>	P53
47	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; <lista_sent> } <lista_sent>	P54
48	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; <sent> <lista_sent> } <lista_sent>	P2
49	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; <sent_cerrada> <lista_sent> } <lista_sent>	P4
50	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; <simple> ; <lista_sent> } <lista_sent>	P6
51	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn <giro> ; <lista_sent> } <lista_sent>	P15
52	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; <lista_sent> } <lista_sent>	P20
53	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } <lista_sent>	P3

54	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } <sent> <lista_sent>	P2
55	=> α1 forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } <sent_cerrada> <lista_sent>	P4
56	=> α2 if <expr> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P8
57	=> α2 if <igualdad> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P33
58	=> α2 if <relacional> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P36
59	=> α2 if <suma> < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P37
60	=> α2 if <termino> < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P44
61	=> α2 if <unario> < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P48
62	=> α2 if <postfijo> < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P51
63	=> α2 if <primario> < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P53
64	=> α2 if IDENT[front_sensor] < <suma> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P55
65	=> α2 if IDENT[front_sensor] < <termino> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P44
66	=> α2 if IDENT[front_sensor] < <unario> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P48
67	=> α2 if IDENT[front_sensor] < <postfijo> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P51
68	=> α2 if IDENT[front_sensor] < <primario> then <sent_cerrada> else <sent_cerrada> <lista_sent>	P53
69	=> α2 if IDENT[front_sensor] < NUMBER[30] then <sent_cerrada> else <sent_cerrada> <lista_sent>	P54
70	=> α2 if IDENT[front_sensor] < NUMBER[30] then <simple> ; else <sent_cerrada> <lista_sent>	P6
71	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else <sent_cerrada> <lista_sent>	P16
72	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else <simple> ; <lista_sent>	P6
73	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <expr> ; <lista_sent>	P14
74	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <igualdad> ; <lista_sent>	P33
75	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <relacional> ; <lista_sent>	P36
76	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <suma> ; <lista_sent>	P41
77	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <termino> ; <lista_sent>	P44
78	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <unario> ; <lista_sent>	P48
79	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <postfijo> ; <lista_sent>	P51
80	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward <primario> ; <lista_sent>	P53
81	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; <lista_sent>	P54
82	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; <sent> <lista_sent>	P2
83	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; <sent_cerrada> <lista_sent>	P4
84	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; <simple> ; <lista_sent>	P6
85	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ; <lista_sent>	P17
86	=> α2 if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P3

Al final, la forma sentencial es exactamente la cadena de terminales de 6.6.

6.8 Programa completo: derivación por la derecha (rightmost)

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent> <sent> <lista_sent>	P2
4	=> <sent> <sent> <sent> <lista_sent>	P2
5	=> <sent> <sent> <sent> <sent> <lista_sent>	P2
6	=> <sent> <sent> <sent> <sent>	P3
7	=> <sent> <sent> <sent> <sent_cerrada>	P4
8	=> <sent> <sent> <sent> <simple> ;	P6
9	=> <sent> <sent> <sent> return base ;	P17
10	=> <sent> <sent> <sent_cerrada> return base ;	P4
11	=> <sent> <sent> if <expr> then <sent_cerrada> else <sent_cerrada> return base ;	P8
12	=> <sent> <sent> if <expr> then <sent_cerrada> else <simple> ; return base ;	P6
13	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <expr> ; return base ;	P14
14	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <igualdad> ; return base ;	P33
15	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <relacional> ; return base ;	P36
16	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <suma> ; return base ;	P41
17	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <termino> ; return base ;	P44
18	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <unario> ; return base ;	P48
19	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <postfijo> ; return base ;	P51
20	=> <sent> <sent> if <expr> then <sent_cerrada> else forward <primario> ; return base ;	P53
21	=> <sent> <sent> if <expr> then <sent_cerrada> else forward NUMBER[1] ; return base ;	P54
22	=> <sent> <sent> if <expr> then <simple> ; else forward NUMBER[1] ; return base ;	P6
23	=> <sent> <sent> if <expr> then stop ; else forward NUMBER[1] ; return base ;	P16
24	=> <sent> <sent> if <igualdad> then stop ; else forward NUMBER[1] ; return base ;	P33
25	=> <sent> <sent> if <relacional> then stop ; else forward NUMBER[1] ; return base ;	P36
26	=> <sent> <sent> if <suma> < <suma> then stop ; else forward NUMBER[1] ; return base ;	P37
27	=> <sent> <sent> if <suma> < <termino> then stop ; else forward NUMBER[1] ; return base ;	P44
28	=> <sent> <sent> if <suma> < <unario> then stop ; else forward NUMBER[1] ; return base ;	P48
29	=> <sent> <sent> if <suma> < <postfijo> then stop ; else forward NUMBER[1] ; return base ;	P51
30	=> <sent> <sent> if <suma> < <primario> then stop ; else forward NUMBER[1] ; return base ;	P53
31	=> <sent> <sent> if <suma> < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P54
32	=> <sent> <sent> if <termino> < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P44
33	=> <sent> <sent> if <unario> < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P48
34	=> <sent> <sent> if <postfijo> < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P51
35	=> <sent> <sent> if <primario> < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P53
36	=> <sent> <sent> if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P55
37	=> <sent> <sent_cerrada> if IDENT[front_sensor] < NUMBER[30] then stop ; else forward NUMBER[1] ; return base ;	P4
38	=> <sent> repeat <expr> times <sent_cerrada> ω1	P9
39	=> <sent> repeat <expr> times <bloque> ω1	P7
40	=> <sent> repeat <expr> times { <lista_sent> } ω1	P13
41	=> <sent> repeat <expr> times { <sent> <lista_sent> } ω1	P2
42	=> <sent> repeat <expr> times { <sent> <sent> <lista_sent> } ω1	P2
43	=> <sent> repeat <expr> times { <sent> <sent> } ω1	P3
44	=> <sent> repeat <expr> times { <sent> <sent_cerrada> } ω1	P4
45	=> <sent> repeat <expr> times { <sent> <simple> ; } ω1	P6
46	=> <sent> repeat <expr> times { <sent> turn <giro> ; } ω1	P15
47	=> <sent> repeat <expr> times { <sent> turn right ; } ω1	P20
48	=> <sent> repeat <expr> times { <sent_cerrada> turn right ; } ω1	P4
49	=> <sent> repeat <expr> times { <simple> ; turn right ; } ω1	P6
50	=> <sent> repeat <expr> times { forward <expr> ; turn right ; } ω1	P14
51	=> <sent> repeat <expr> times { forward <igualdad> ; turn right ; } ω1	P33
52	=> <sent> repeat <expr> times { forward <relacional> ; turn right ; } ω1	P36
53	=> <sent> repeat <expr> times { forward <suma> ; turn right ; } ω1	P41

54	=> <sent> repeat <expr> times { forward <suma> + <termino> ; turn right ; } ω1	P42
55	=> <sent> repeat <expr> times { forward <suma> + <unario> ; turn right ; } ω1	P48
56	=> <sent> repeat <expr> times { forward <suma> + <postfijo> ; turn right ; } ω1	P51
57	=> <sent> repeat <expr> times { forward <suma> + <primario> ; turn right ; } ω1	P53
58	=> <sent> repeat <expr> times { forward <suma> + NUMBER[5] ; turn right ; } ω1	P54
59	=> <sent> repeat <expr> times { forward <termino> + NUMBER[5] ; turn right ; } ω1	P44
60	=> <sent> repeat <expr> times { forward <termino> * <unario> + NUMBER[5] ; turn right ; } ω1	P45
61	=> <sent> repeat <expr> times { forward <termino> * <postfijo> + NUMBER[5] ; turn right ; } ω1	P51
62	=> <sent> repeat <expr> times { forward <termino> * <primario> + NUMBER[5] ; turn right ; } ω1	P53
63	=> <sent> repeat <expr> times { forward <termino> * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P55
64	=> <sent> repeat <expr> times { forward <unario> * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P48
65	=> <sent> repeat <expr> times { forward <postfijo> * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P51
66	=> <sent> repeat <expr> times { forward <primario> * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P53
67	=> <sent> repeat <expr> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P54
68	=> <sent> repeat <igualdad> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P33
69	=> <sent> repeat <relacional> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P36
70	=> <sent> repeat <suma> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P41
71	=> <sent> repeat <termino> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P44
72	=> <sent> repeat <unario> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P48
73	=> <sent> repeat <postfijo> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P51
74	=> <sent> repeat <primario> times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P53
75	=> <sent> repeat NUMBER[4] times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P54
76	=> <sent_cerrada> repeat NUMBER[4] times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P4
77	=> <simple> ; repeat NUMBER[4] times { forward NUMBER[10] * IDENT[speed] + NUMBER[5] ; turn right ; } ω1	P6
78	=> IDENT[speed] = <expr> ; ω2	P18
79	=> IDENT[speed] = <igualdad> ; ω2	P33
80	=> IDENT[speed] = <relacional> ; ω2	P36
81	=> IDENT[speed] = <suma> ; ω2	P41
82	=> IDENT[speed] = <termino> ; ω2	P44
83	=> IDENT[speed] = <unario> ; ω2	P48
84	=> IDENT[speed] = <postfijo> ; ω2	P51
85	=> IDENT[speed] = <primario> ; ω2	P53
86	=> IDENT[speed] = NUMBER[2] ; ω2	P54

Al final, la forma sentencial corresponde a la misma cadena de terminales. Las dos derivaciones constan de 86 pasos y emplean las mismas producciones en distinto orden.

6.9 Árbol de derivación del programa completo

Las dos derivaciones anteriores construyen el mismo árbol. Dado que es muy ancho y profundo para una página, se presenta en partes: el nivel superior (Figura 10), con las cuatro sentencias resumidas en recuadros de esquinas redondeadas, y luego el subárbol completo de cada una (Figuras 11 a 15); dentro de ellos, las expresiones también se resumen en recuadros y se desarrollan aparte.

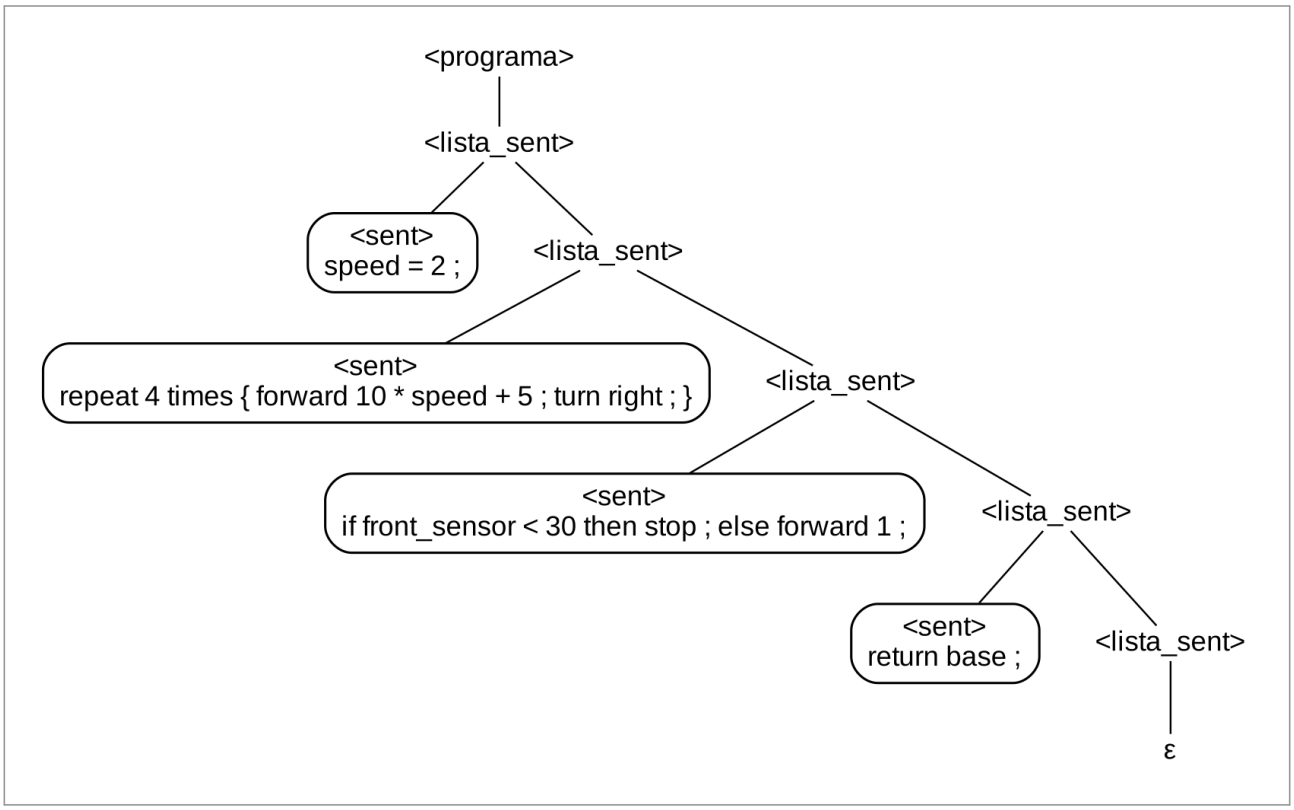


Figura 10. Nivel superior del árbol del programa completo. Cada recuadro de <sent> se desarrolla en las Figuras 11 a 14.

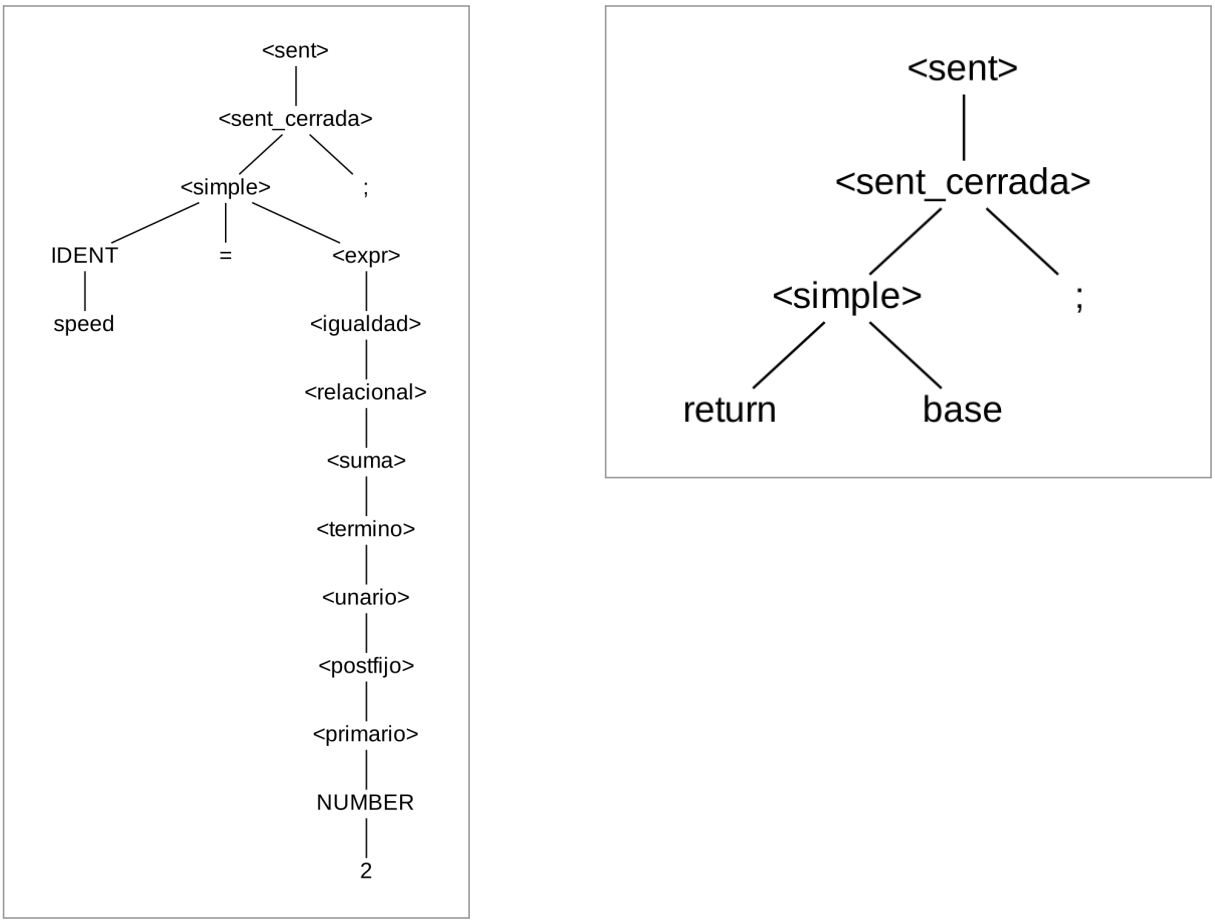


Figura 11. Subárboles de la asignación `speed = 2;` (izquierda) y de `return base;` (derecha). Un número como 2 se deriva con la misma cadena <expr> → … → NUMBER.

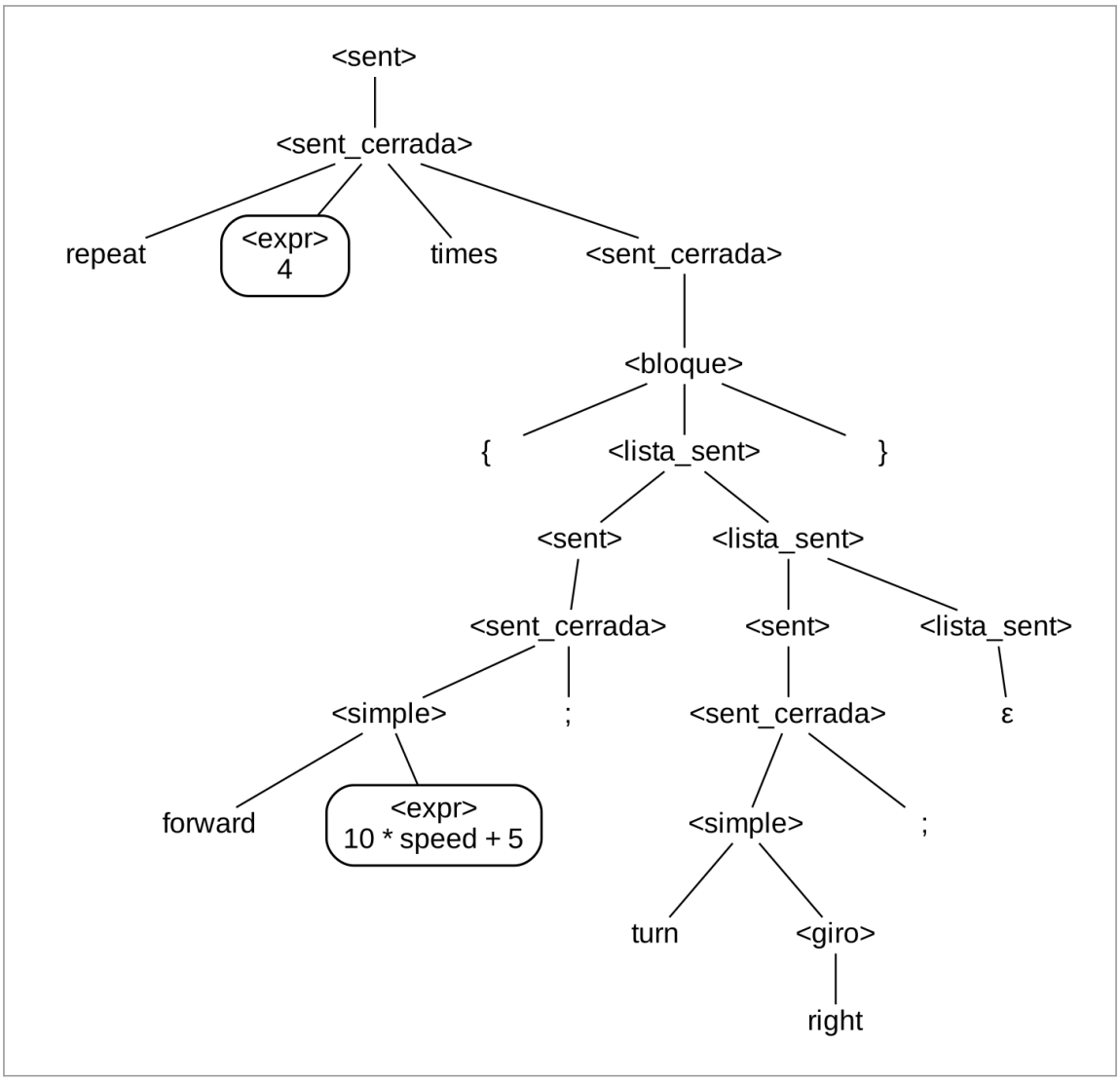


Figura 12. Subárbol de <sent> para `repeat 4 times { forward 10 * speed + 5; turn right; }`. El recuadro de la `forward` se desarrolla en la Figura 13.

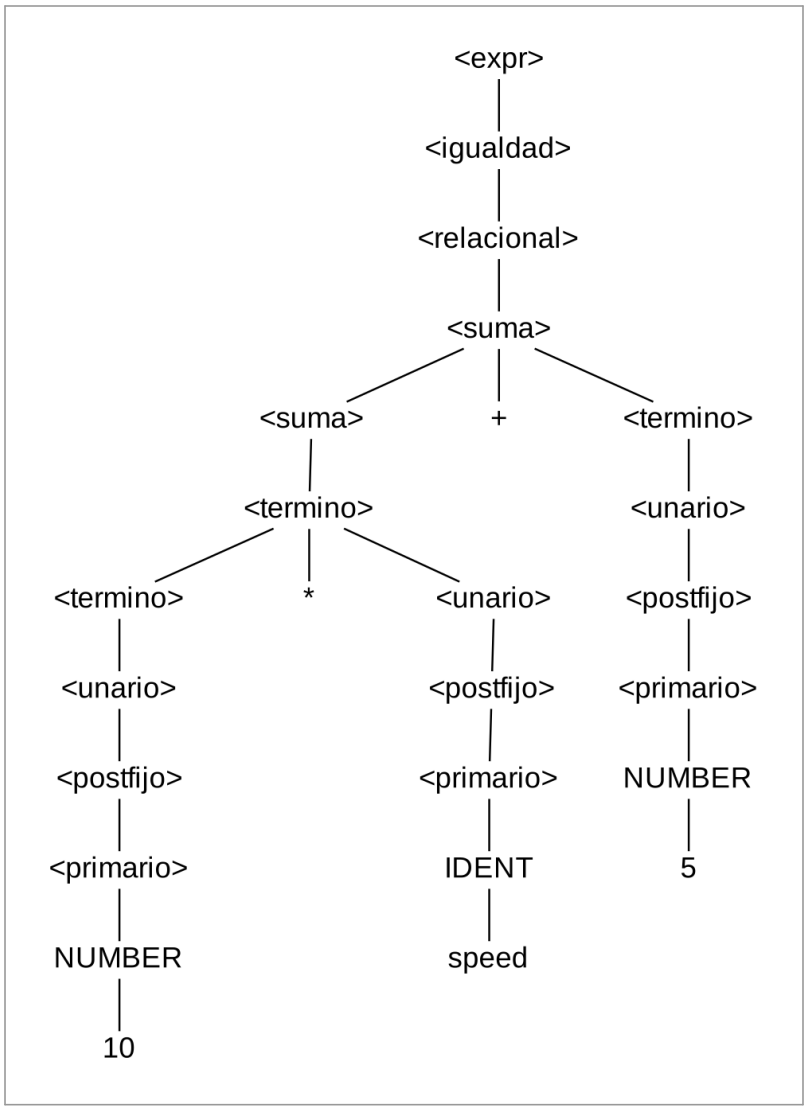


Figura 13. Subárbol de la expresión `10 * speed + 5`.

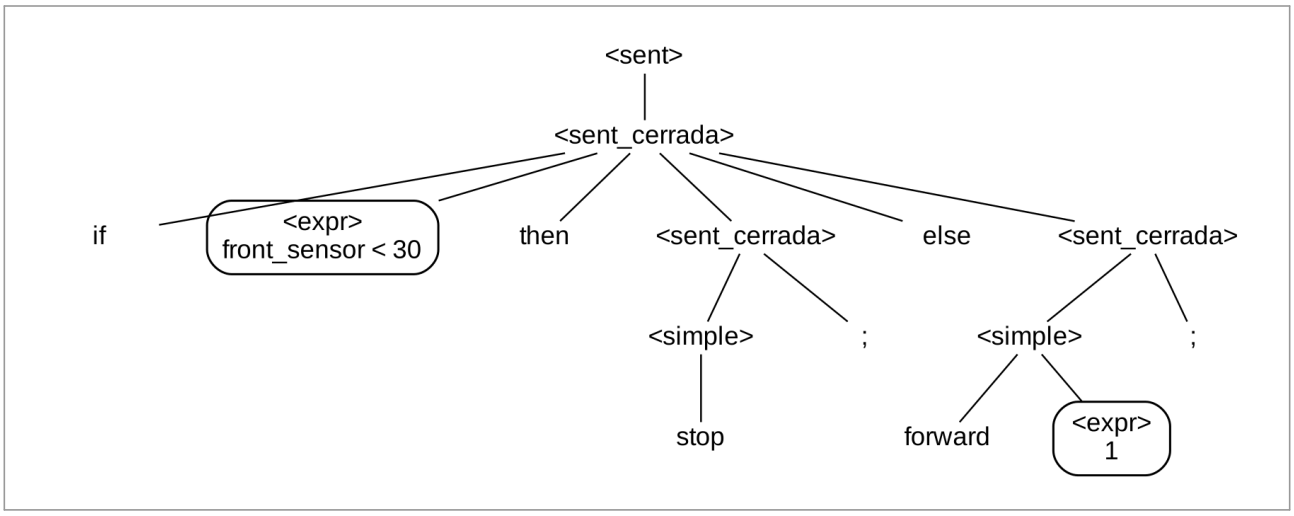


Figura 14. Subárbol de la decisión `if front_sensor < 30 then stop; else forward 1;`. La condición se desarrolla en la Figura 15.

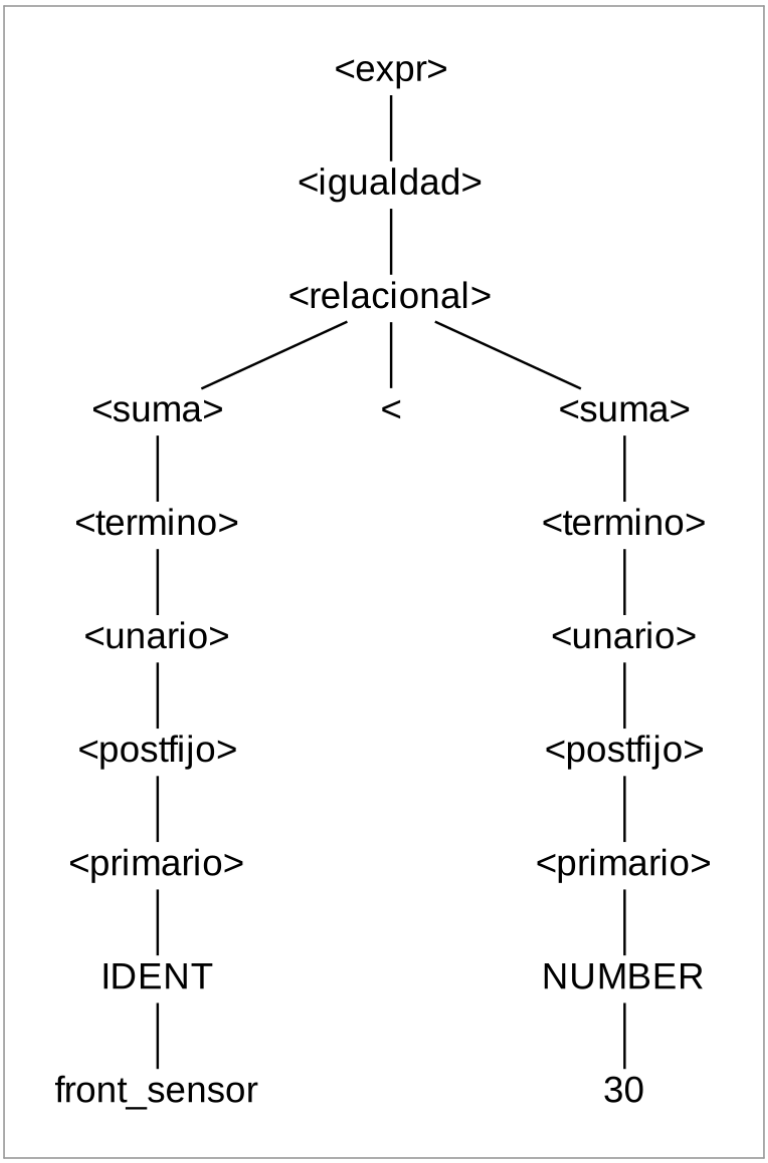


Figura 15. Subárbol de la condición `front_sensor < 30`: la comparación queda en <relacional>.

7. Programas mal escritos y punto en el que la gramática los rechaza

Un programa está mal escrito cuando su cadena de tokens no puede derivarse desde <programa>. Para encontrar el punto exacto del rechazo se deriva por la izquierda, leyendo los tokens en orden, hasta que el siguiente token no coincide con lo que la gramática exige.

7.1 Primer ejemplo: falta la palabra times

```
repeat 4 {
    forward 10;
}
```

Cadena de terminales: `repeat NUMBER { forward NUMBER ; }`

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent_cerrada> <lista_sent>	P4
4	=> repeat <expr> times <sent_cerrada> <lista_sent>	P9
5	=> repeat <igualdad> times <sent_cerrada> <lista_sent>	P33
6	=> repeat <relacional> times <sent_cerrada> <lista_sent>	P36
7	=> repeat <suma> times <sent_cerrada> <lista_sent>	P41
8	=> repeat <termino> times <sent_cerrada> <lista_sent>	P44
9	=> repeat <unario> times <sent_cerrada> <lista_sent>	P48
10	=> repeat <postfijo> times <sent_cerrada> <lista_sent>	P51
11	=> repeat <primario> times <sent_cerrada> <lista_sent>	P53
12	=> repeat NUMBER[4] times <sent_cerrada> <lista_sent>	P54

Punto de rechazo: se exige `times` y el siguiente token es `{`; ninguna producción permite continuar.

Las dos reglas para `repeat` (P9 y P12) exigen `times` después de la expresión, pero el siguiente token es `{`. La gramática deja de aceptar exactamente en este punto. El mensaje que entrega el procesador a quien escribió el programa es:

```
error[sintáctico] línea 1, columna 10: se esperaba 'times' después del número de repeticiones pero se
encontró '{'
  1 | repeat 4 {
    |          ^
  ayuda: Sintaxis: repeat <expresión> times <sentencia>
```

Corrección: escribir `repeat 4 times {` en la línea 1.

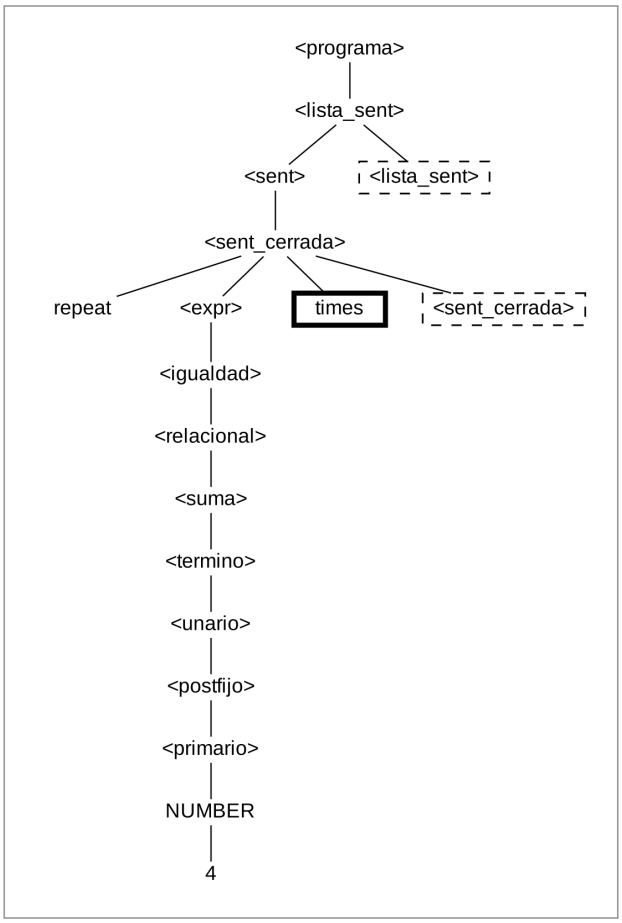


Figura 16. Árbol de derivación hasta el punto del error. El recuadro de borde grueso es el terminal que se exige y no aparece; los de borde punteado quedaron pendientes.

7.2 Segundo ejemplo: a un número no se le puede sumar «stop»

```
repeat 3 times {
    forward 5 + stop;
}
```

Cadena de terminales: `repeat NUMBER times { forward NUMBER + stop ; }`

A la suma le falta el segundo operando: después del `+` aparece `stop`, que es una palabra reservada. Se deriva por la izquierda hasta llegar a la suma:

	<programa>	
1	=> <lista_sent>	P1
2	=> <sent> <lista_sent>	P2
3	=> <sent_cerrada> <lista_sent>	P4
4	=> repeat <expr> times <sent_cerrada> <lista_sent>	P9
5	=> repeat <igualdad> times <sent_cerrada> <lista_sent>	P33
6	=> repeat <relacional> times <sent_cerrada> <lista_sent>	P36
7	=> repeat <suma> times <sent_cerrada> <lista_sent>	P41
8	=> repeat <termino> times <sent_cerrada> <lista_sent>	P44
9	=> repeat <unario> times <sent_cerrada> <lista_sent>	P48
10	=> repeat <postfijo> times <sent_cerrada> <lista_sent>	P51
11	=> repeat <primario> times <sent_cerrada> <lista_sent>	P53
12	=> repeat NUMBER[3] times <sent_cerrada> <lista_sent>	P54
13	=> repeat NUMBER[3] times <bloque> <lista_sent>	P7
14	=> repeat NUMBER[3] times { <lista_sent> } <lista_sent>	P13
15	=> repeat NUMBER[3] times { <sent> <lista_sent> } <lista_sent>	P2
16	=> repeat NUMBER[3] times { <sent_cerrada> <lista_sent> } <lista_sent>	P4
17	=> repeat NUMBER[3] times { <simple> ; <lista_sent> } <lista_sent>	P6
18	=> repeat NUMBER[3] times { forward <expr> ; <lista_sent> } <lista_sent>	P14
19	=> repeat NUMBER[3] times { forward <igualdad> ; <lista_sent> } <lista_sent>	P33
20	=> repeat NUMBER[3] times { forward <relacional> ; <lista_sent> } <lista_sent>	P36
21	=> repeat NUMBER[3] times { forward <suma> ; <lista_sent> } <lista_sent>	P41
22	=> repeat NUMBER[3] times { forward <suma> + <termino> ; <lista_sent> } <lista_sent>	P42
23	=> repeat NUMBER[3] times { forward <termino> + <termino> ; <lista_sent> } <lista_sent>	P44
24	=> repeat NUMBER[3] times { forward <unario> + <termino> ; <lista_sent> } <lista_sent>	P48
25	=> repeat NUMBER[3] times { forward <postfijo> + <termino> ; <lista_sent> } <lista_sent>	P51
26	=> repeat NUMBER[3] times { forward <primario> + <termino> ; <lista_sent> } <lista_sent>	P53
27	=> repeat NUMBER[3] times { forward NUMBER[5] + <termino> ; <lista_sent> } <lista_sent>	P54

Punto de rechazo: debe derivarse <termino> y el siguiente token es `stop`; ninguna producción permite continuar.

En este punto, el siguiente token de la entrada es `stop`, pero lo que corresponde derivar es <termino>. <termino> solo puede producir <termino> * <unario>, <termino> / <unario>, <termino> % <unario> o <unario> (P45–P48), y <unario> solo puede comenzar con -, +, un NUMBER, un IDENT o ((P49–P56). La palabra `stop` no es ninguna de esas opciones. Es aquí donde la gramática rechaza el programa. La Figura 17 muestra el árbol hasta ese punto. El mensaje del procesador es:

```
error[sintáctico] línea 2, columna 17: se esperaba una expresión (número, variable o '(') pero se
encontró 'stop'
  2 |     forward 5 + stop;
    |                 ^
  ayuda: Ejemplos de expresión: 10, speed, 10 * speed + 5, (a + b) * 2
```

Corrección: `forward 5 + 2;`. Después del + corresponde un NUMBER, que es lo que <termino> puede producir, por lo que ese programa sí es aceptado por la gramática (el robot avanzaría 7, tres veces).

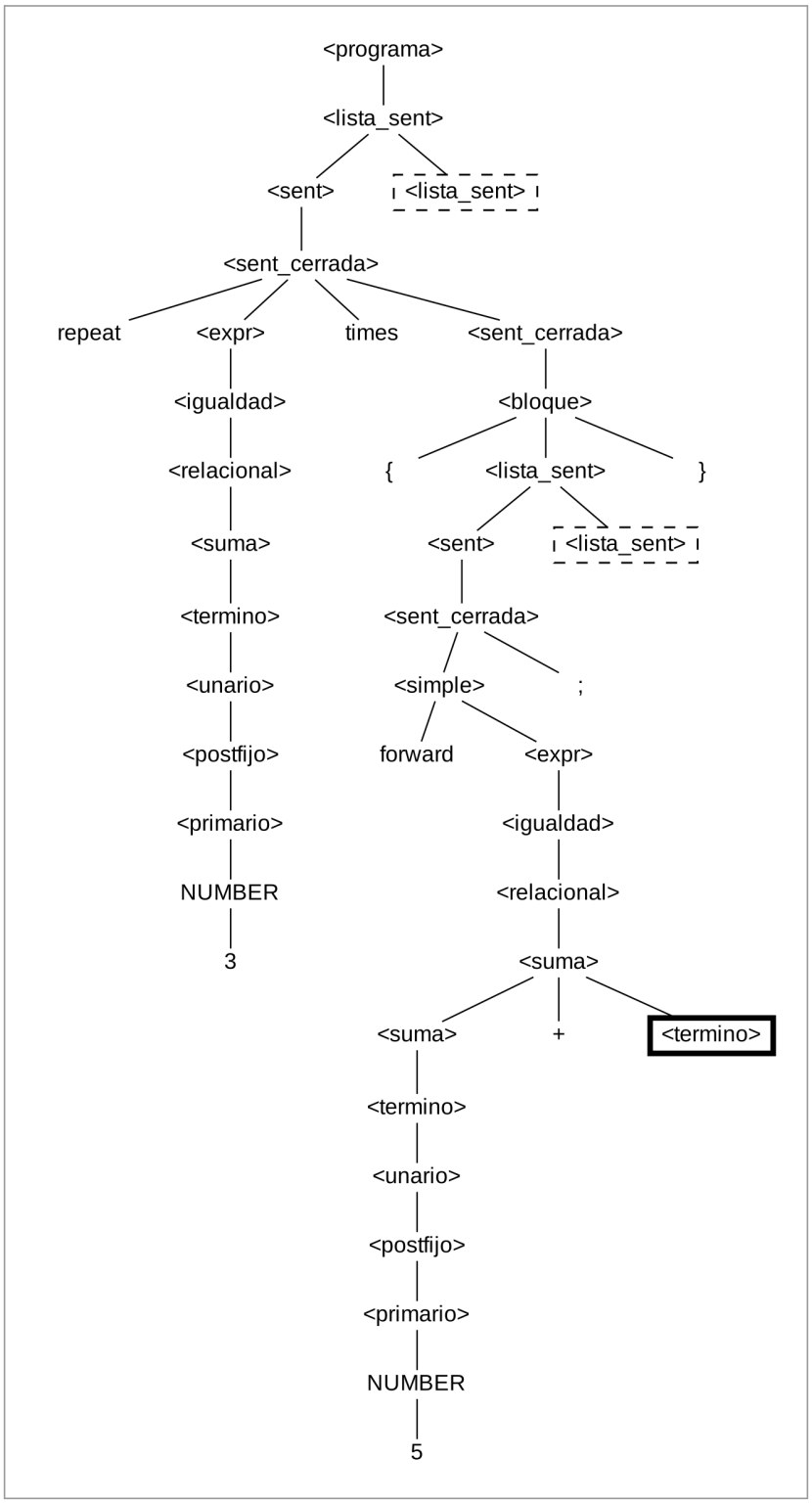


Figura 17. Árbol de derivación hasta el punto del error. El recuadro de borde grueso es el nodo que no puede desarrollarse; los de borde punteado quedaron pendientes.

7.3 Otros casos

Programa erróneo	Punto de rechazo	Mensaje del procesador	Corrección
`speed = 3` `forward 1;`	Después de IDENT = <expr> se exige ; (P6) o un operador, y llega `forward`.	Línea 1, col. 10: falta ';' al final de la sentencia	Agregar `;` al final de la línea 1.
`if speed > 2 forward 1;`	Después de la condición se exige `then` (P8, P10 y P11) y llega `forward`.	Línea 1, col. 14: se esperaba 'then' después de la condición del 'if' pero se encontró 'forward'	`if speed > 2 then forward 1;`
`forward 1;` `else stop;`	Ninguna producción de <sent> comienza con `else`; solo puede aparecer después de un `then`.	Línea 2, col. 1: 'else' sin un 'if' correspondiente	Quitar el `else` o agregar el `if` que le corresponde.
`repeat 2 times {` `  forward 1;`	<bloque> exige } (P13) y llega el fin del archivo.	Línea 1, col. 16: la llave '{' abierta aquí nunca se cierra	Agregar } al final.
`turn up;`	Después de `turn` se exige right o left (P20 y P21) y llega un IDENT.	Línea 1, col. 6: después de 'turn' se esperaba 'right' o 'left' pero se encontró 'up'	`turn right;`
`return home;`	P17 exige `base` después de `return`.	Línea 1, col. 8: después de 'return' se esperaba 'base' pero se encontró 'home'	`return base;`
`x = (2 * speed;`	El paréntesis abierto exige) (P56) y llega `;`.	Línea 1, col. 5: el paréntesis '(' abierto aquí no se cierra	`x = (2 * speed);`
`x = 1 < 2 < 3;`	<relacional> admite una sola comparación (P37–P40): después de `1 < 2` no puede venir otro <.	Línea 1, col. 11: '<' y '<' no son asociativos entre sí (no se pueden encadenar)	Usar paréntesis: `(1 < 2) < 3`.
`x = 4 @ 2;`	@ no coincide con ningún patrón: falla el analizador léxico, antes que el sintáctico.	Línea 1, col. 7: carácter no reconocido '@'	Quitar el @ o usar un operador válido: `x = 4 + 2;`

Un error léxico consiste en un símbolo que no es ningún token; un error sintáctico consiste en tokens válidos dispuestos en un orden incorrecto. Distinguirlos permite emitir mensajes más claros. Después de un error sintáctico el procesador no se detiene: se resincroniza en el siguiente `;`, llave o palabra clave de sentencia y continúa, de modo que informa varios errores en una sola pasada.

8. Vocabulario de Ruta: tokens, patrones y lexemas

Antes de que un programa pueda revisarse con la gramática, debe pasar por el analizador léxico. Su función consiste en leer el programa carácter por carácter y separarlo en «palabras» con significado (Aho et al., 2007, cap. 3). En esta sección se define qué tokens tiene Ruta, qué patrón describe a cada uno, a qué terminal de la gramática corresponde y cómo queda un programa real después del análisis léxico. Se emplean tres conceptos (Morales, s. f.-a):

- Lexema: la palabra exacta que aparece en el programa (ejemplo: speed).
- Token: la categoría a la que pertenece ese lexema (ejemplo: IDENT, identificador).
- Patrón: la regla que establece qué lexemas pertenecen a cada token (ejemplo: «empieza con una letra o _ y continúa con letras, dígitos o _»).

Notación de los patrones: letra (a–z, A–Z), dígito (0–9), | (alternativa), * (cero o más), + (uno o más), ? (opcional).

Tabla 10. Tokens de Ruta

Token	Patrón	Terminal en la gramática	Lexemas de ejemplo
Palabra reservada (FORWARD, TURN, RIGHT, LEFT, STOP, RETURN, BASE, IF, THEN, ELSE, REPEAT, TIMES, OPERATOR, INFIX, PREFIX, POSTFIX, PRECEDENCE, ASSOC)	La palabra misma: `forward turn right left stop return base if then else repeat times operator infix prefix postfix precedence assoc`	Cada palabra es su propio terminal	repeat, if, base
NUMBER (número)	`dígito+ ( . dígito+ )?`	NUMBER	2, 30, 3.14
IDENT (identificador)	`(letra | _) (letra | dígito | _)*`, excepto las palabras reservadas	IDENT	speed, front_sensor, _tmp
OP (operador)	Secuencia maximal de caracteres de `+ - * / % < > = ! & | ^ ~ ?`; no es la secuencia = sola y se interrumpe donde comienza un comentario (`//` o `/*`)	Cada operador usado en las expresiones es su propio terminal (`+ - * / % < > <= >= == != !`); en `operator`, el terminal OP	*, +, <=, ==, !, **
ASSIGN (asignación)	=	=	=
LPAREN, RPAREN	`\( | \)`	Cada símbolo es su propio terminal	( )

Token	Patrón	Terminal en la gramática	Lexemas de ejemplo
LBRACE, RBRACE	`\{ | \}`	Cada símbolo es su propio terminal	`{ }`
SEMI (punto y coma)	`;`	`;`	`;`
Comentario de línea (se descarta)	`// (cualquier carácter menos salto de línea)*`	— (no llega al parser)	`// head back to base`
Comentario de bloque (se descarta)	`/* … */`	— (no llega al parser)	`/* nota */`
Espacio (se descarta)	(espacio | tabulación | salto de línea)+	— (no llega al parser)	
EOF	fin de la entrada	—	

El analizador léxico entrega al sintáctico pares (token, lexema). Para IDENT y NUMBER, el parser solo considera la clase del token; para las palabras reservadas, los operadores y los delimitadores, el lexema determina cuál es el terminal (por ejemplo, el token OP con lexema * corresponde al terminal *). Además, cada token OP conserva si tenía espacio a su izquierda y a su derecha, información que usa la sección 9.

Reglas del analizador léxico

Algunos lexemas podrían pertenecer a más de un token. Para resolverlo, el analizador léxico aplica dos reglas (Aho et al., 2007, cap. 3):

- Las palabras reservadas tienen prioridad. Por ejemplo, `repeat` cumple con el patrón de identificador, pero como figura en la lista de palabras reservadas se clasifica como palabra reservada. En cambio, `front_sensor` es un identificador porque no figura en esa lista (y `forwardx` también, por la regla siguiente).
- Máxima coincidencia: se elige el lexema más largo posible. Si aparece <=, podría leerse como un < seguido de un = (2 tokens) o como <= (1 token). Dado que <= existe en el lenguaje y es más largo, se elige esa opción. Por la misma razón, una secuencia de símbolos de operador es *un solo* token: `x=-1` se lee `x`, `=-`, `1`, y como `=-` no es =, la asignación no se reconoce (se escribe `x = -1;`).

Además, un número pegado a letras o a _ (como `2x`) se reporta como lexema mal formado en lugar de separarse en 2 y x, porque casi siempre corresponde a un error de escritura. Los comentarios y los espacios se descartan.

Salida del analizador léxico para el programa del enunciado

Corresponde al programa de la sección 6.6. Al pasar por el analizador léxico se obtienen los siguientes lexemas, con su token y su terminal:

#	Lexema	Token	Terminal	Línea:col.
1	speed	IDENT	IDENT	1:1
2	=	ASSIGN	=	1:7
3	2	NUMBER	NUMBER	1:9
4	;	SEMI	;	1:10
5	repeat	REPEAT	repeat	2:1
6	4	NUMBER	NUMBER	2:8
7	times	TIMES	times	2:10
8	{	LBRACE	{	2:16
9	forward	FORWARD	forward	3:5
10	10	NUMBER	NUMBER	3:13
11	*	OP	*	3:16
12	speed	IDENT	IDENT	3:18
13	+	OP	+	3:24
14	5	NUMBER	NUMBER	3:26
15	;	SEMI	;	3:27
16	turn	TURN	turn	4:5
17	right	RIGHT	right	4:10
18	;	SEMI	;	4:15
19	}	RBRACE	}	5:1
20	if	IF	if	6:1
21	front_sensor	IDENT	IDENT	6:4
22	<	OP	<	6:17
23	30	NUMBER	NUMBER	6:19
24	then	THEN	then	6:22
25	stop	STOP	stop	7:5
26	;	SEMI	;	7:9
27	else	ELSE	else	8:1
28	forward	FORWARD	forward	9:5
29	1	NUMBER	NUMBER	9:13
30	;	SEMI	;	9:14
31	return	RETURN	return	11:1

#	Lexema	Token	Terminal	Línea:col.
32	`base`	BASE	`base`	11:8
33	`;`	SEMI	`;`	11:12

Se obtienen 33 tokens (además del fin de archivo). El comentario de la línea 10 y los espacios o saltos de línea no aparecen porque el analizador léxico los descarta. El lexema `speed` aparece dos veces: se trata de dos tokens de la misma categoría (IDENT) con el mismo lexema. La columna «Terminal», leída de arriba hacia abajo, es exactamente la cadena de terminales que se derivó en las secciones 6.7 y 6.8.

Errores léxicos

El analizador léxico también detecta errores antes de que el programa llegue a la gramática: ocurren cuando encuentra algo que no coincide con ningún patrón. Por ejemplo:

- `x = 4 @ 2;` — el «@» no coincide con ningún patrón (carácter no reconocido); tampoco lo hacen `$`, `"` ni `.` suelto.
- `x = 2x;` — lexema mal formado: un nombre no puede comenzar con un dígito.
- `/* nota sin cerrar` — comentario de bloque sin cerrar: falta `*/`.

En cada caso el procesador informa la línea y la columna y sigue analizando; el lexema inválido llega al parser como un token INVALID que solo sirve para resincronizar y no genera un segundo mensaje.

9. Operadores según el espaciado y precedencia declarable (inspirado en Swift)

El procesador de la Parte 2 está escrito en Swift y Ruta imita dos ideas de ese lenguaje. En Swift la gramática base no fija la precedencia de los operadores: la define el programador (`precedencegroup`), y el espaciado decide si un operador es prefijo, infijo o postfijo. Esto cambia la manera de describir la sintaxis de las expresiones, y por eso se explica aquí aparte de la gramática estática de las secciones 3 y 4.

9.1 Fijación por espaciado (analizador léxico)

Cada token OP guarda si tenía espacio a su izquierda y a su derecha. «Espacio» incluye un espacio en blanco, un comentario, el inicio o el fin del archivo, y la puntuación de apertura (`(` `{` `,` `;`) a la izquierda o de cierre (`)` `}` `,` `;`) a la derecha. La clasificación es la misma de Swift:

Tabla 11. Fijación de un operador según sus espacios

Escritura	Espacios izq./der.	Fijación	Significado
`a + b`	sí / sí	infijo	suma
`a+b`	no / no	infijo (apretado)	suma
`a +b`	sí / no	prefijo	«a» seguido de «+b»: error, falta un operador
`a+ b`	no / sí	postfijo	«a+» seguido de «b»: error, «+» no es postfijo
`-x`, `(-x)`, `= -x`	sí / no	prefijo	negación
`n!;`, `n! + 1`	no / sí	postfijo	factorial

Ejemplos con el procesador. `x = 5 +3;` se rechaza con «el operador '+' tiene espacio a la izquierda pero no a la derecha, por lo que se lee como PREFIJO y aquí falta un operador entre dos operandos», y y `= 5+ 3;` con «el operador '+' tiene espacio solo a la derecha […], lo que lo hace POSTFIJO, y no está definido como postfijo».

9.2 Precedencia en una tabla, no en la gramática (analizador sintáctico)

Con precedencia declarable, la gramática ya no puede decir qué operador liga más fuerte: `Parser.parseExpr` es un analizador de precedencias (Pratt) que consulta una tabla de operadores. El programa puede **redeclarar** la tabla en cualquier punto, y el cambio vale desde allí hasta el final del archivo:

```
operator ** infix precedence 80 assoc right;   // 2 ** 3 ** 2 = 2 ** 9
operator + infix precedence 90;                // ahora + liga más que *
```

Esa declaración es una instrucción más de la gramática (<decl_op>, P22–P32). En esa versión «dinámica», la EBNF de las expresiones es plana:

expr = unario { OP_infijo unario } ;
unario = OP_prefijo unario | primario { OP_postfijo } ;
primario = NUMBER | IDENT | "(" expr ")" ;

Aquí la gramática no establece qué operador va primero: lo decide la tabla. La gramática estratificada de la sección 3 es la versión «estática» equivalente cuando la tabla es la de fábrica (Tabla 12) y el espaciado no es significativo.

Tabla 12. Operadores de fábrica

Posición	Operadores	Precedencia / asociatividad
infijo	`* / %`	70, izquierda
infijo	`+ -`	60, izquierda
infijo	`< > <= >=`	40, no asociativo
infijo	`== !=`	30, no asociativo
prefijo	`- +`	ligan más que cualquier infijo
postfijo	`!` (factorial)	ligan más que prefijo

9.3 Precedencia por espaciado

Un infijo **apretado** (a+b, sin espacios) recibe un bonus de +1000 sobre uno con espacios, de modo que el espaciado visual coincide con la agrupación:

Tabla 13. Efecto del espaciado en la agrupación

Fuente	Agrupación	Valor
`1 + 2*3`	`1 + (2*3)`	7
`1+2 * 3`	`(1+2) * 3`	9
`10 * speed + 5`	`(10 * speed) + 5`	35 con speed = 3 (normal)

Esta regla se desactiva con la opción `--strict` del procesador (`swift run ruta prog.ruta --strict`), en cuyo caso solo manda la tabla. Las gramáticas de las secciones 3 y 4 describen el lenguaje con el espaciado uniforme (todos los operadores con espacio a ambos lados o ninguno), que es el caso de todos los ejemplos de este documento.

10. Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, techniques, and tools (2.ª ed.). Pearson / Addison-Wesley. [Edición en español: Compiladores: principios, técnicas y herramientas, Pearson Educación, 2008.]

ISO/IEC. (1996). ISO/IEC 14977:1996. Information technology — Syntactic metalanguage — Extended BNF. International Organization for Standardization.

Louden, K. C. y Lambert, K. A. (2011). Programming languages: Principles and practice (3.ª ed.). Cengage Learning.

Morales, H. (s. f.-a). Procesadores de lenguaje: gramáticas libres de contexto, BNF y EBNF, árboles de derivación, gramáticas ambiguas y análisis léxico [Diapositivas de clase]. Ingeniería de Sistemas.

Morales, H. (s. f.-b). Graded workshop: Ruta, a language for the campus robot [Enunciado del taller calificado]. Procesadores de Lenguaje, Ingeniería de Sistemas.

Naur, P. (Ed.). (1963). Revised report on the algorithmic language ALGOL 60. Communications of the ACM, 6(1), 1–17.

Sipser, M. (2013). Introduction to the theory of computation (3.ª ed.). Cengage Learning.

Wirth, N. (1977). What can we do about the unnecessary diversity of notation for syntactic definitions? Communications of the ACM, 20(11), 822–823.